



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ
ΕΡΓΑΣΤΗΡΙΟ ΣΥΣΤΗΜΑΤΩΝ ΒΑΣΕΩΝ ΓΝΩΣΕΩΝ ΚΑΙ ΔΕΔΟΜΕΝΩΝ
Ακ. έτος 2025-2026, 6ο εξάμηνο, ΣΗΜΜΥ
Βάσεις Δεδομένων - Εξαμηνιαία Εργασία

Η παρούσα αναφορά συντάχθηκε στα πλαίσια της εξαμηνιαίας εργασίας του μαθήματος «Βάσεις Δεδομένων». Σκοπός της εργασίας είναι η σχεδίαση και υλοποίηση ενός συστήματος αποθήκευσης και διαχείρισης πληροφοριών για το Γενικό Νοσοκομείο «Υγειόπολης».

Δημήτριος Θεοδωρίδης --- 03123801

Διονύσιος Βάκουλης --- 03123031

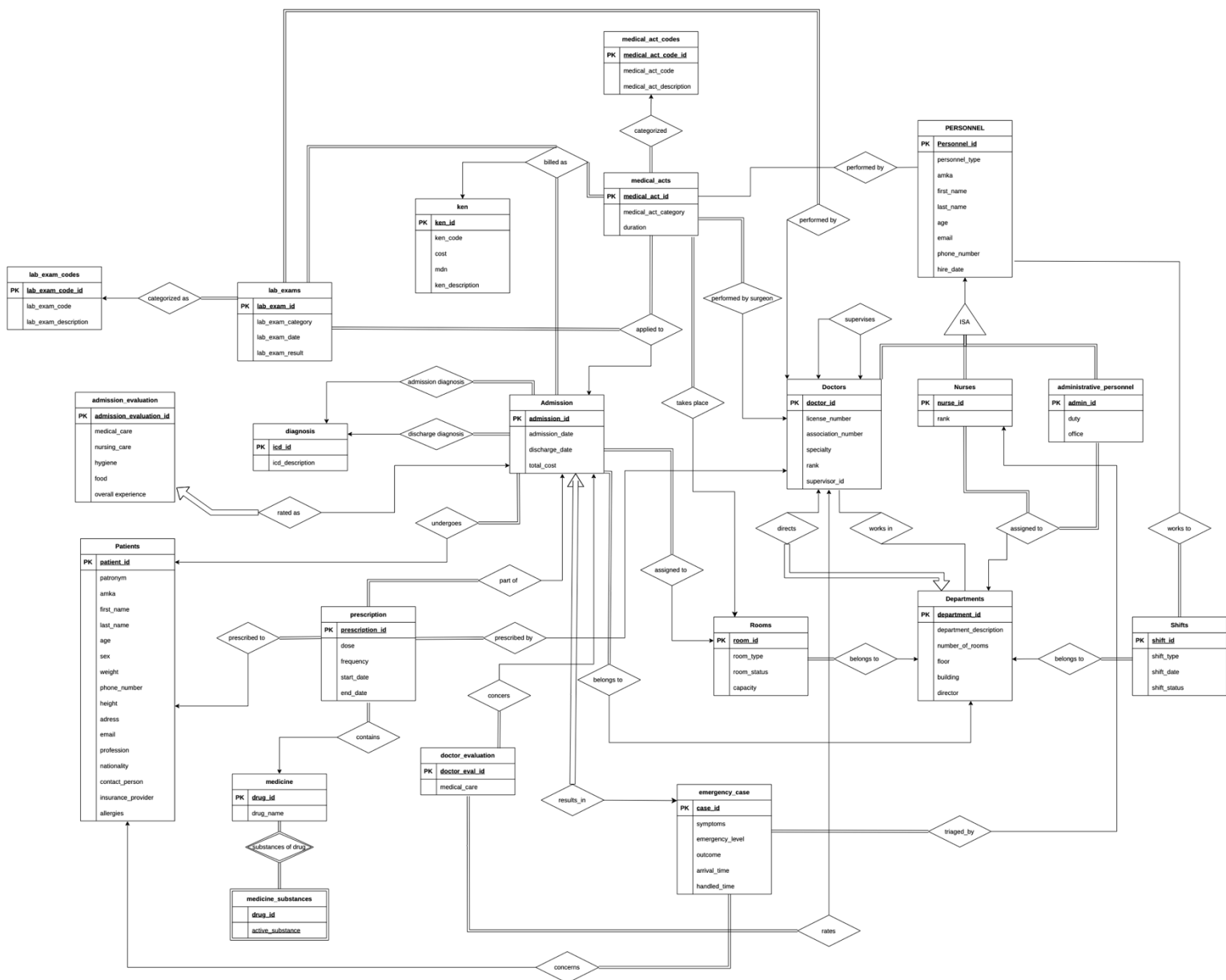
Λάμπρος Χουτόπουλος --- 03123914

Περιεχόμενα

A. Σχεδιασμός και Υλοποίηση Βάσης Δεδομένων.....	3
A.1 Διάγραμμα E-R	3
A.2 Υλοποίηση Βάσης Δεδομένων.....	9
A.2.1 Σχεσιακό Διάγραμμα.....	9
A.2.2 Περιορισμοί Ορθότητας.....	10
A.2.3 Συναρτήσεις και Διαδικασίες	12
A.3 Mock Data.....	15
B. Queries.....	16

A. Σχεδιασμός και Υλοποίηση Βάσης Δεδομένων

A.1 Διάγραμμα E-R



Το παραπάνω διάγραμμα περιέχει όλες τις οντότητες της βάσης, συμπεριλαμβανομένων των επιμέρους χαρακτηριστικών τους και των σχέσεων που τις συνδέει. Ειδικότερα:

- **Personnel – Doctors**

Κάθε εγγραφή στον πίνακα doctors αντιστοιχεί σε μία μοναδική εγγραφή στον πίνακα personnel. Επομένως, η σχέση είναι 1:1 και παρουσιάζει ολική συμμετοχή από την πλευρά των doctors. Ο πίνακας doctors επεκτείνει τον personnel προσθέτοντας τα ιατρικά στοιχεία των ιατρών.

- **Personnel – Nurses**

Αντίστοιχα με τους ιατρούς, η σχέση μεταξύ των δύο οντοτήτων είναι 1:1 και ο πίνακας nurses αποθηκεύει επιπλέον πληροφορίες για τους νοσηλευτές.

- **Personnel – Administrative Personnel**

Με την ίδια λογική και σχέση υλοποιείται ο πίνακας των διοικητικών υπαλλήλων.

- **Doctors – Doctors**

Μερικοί ιατροί, ανάλογα με την βαθμίδα τους, ενδέχεται να έχουν κάποιον άλλον ιατρό ως επόπτη. Ένας ιατρός μπορεί να είναι επόπτης πολλών άλλων κατώτερων ιατρών, άλλα ένας ιατρός μπορεί να έχει έναν μοναδικό επόπτη. Επομένως, υπάρχει μια αυτό-αναφορά, η οποία αναπαριστά την ιεραρχική δομή των ιατρών. Η σχέση είναι τύπου 1:N, με την πλευρά του επόπτη να είναι 1 και την πλευρά των εποπτευόμενων να είναι N. Ο επόπτης μπορεί να είναι και NULL, γεγονός που είναι απαραίτητο στους ιατρούς της ανώτερης βαθμίδας (διευθυντής).

- **Doctors – Departments**

1. **Σχέση Διοίκησης**

Ένας διευθυντής ιατρός μπορεί να διευθύνει ένα ή περισσότερα τμήματα και ένα τμήμα έχει αυστηρά έναν διευθυντή (total participation). Επομένως η σχέση είναι 1:N με το N στην πλευρά των τμημάτων.

2. **Σχέση Στελέχωσης**

Ένας ιατρός μπορεί να ανήκει σε ένα ή περισσότερα τμήματα και προφανώς σε ένα τμήμα υπηρετούν πολλοί ιατροί. Η σχέση είναι M:N

- **Nurses – Departments**

Κάθε τμήμα απασχολεί πολλούς νοσηλευτές και κάθε νοσηλευτής ανήκει σε ακριβώς ένα σχήμα. Η σχέση είναι 1:N και παρουσιάζει ολική συμμετοχή από την πλευρά των νοσηλευτών.

- **Administrative Personnel – Departments**

Υφίσταται όμοια σχέση με την Nurses – Departments.

- **Shifts – Departments**

Ένα τμήμα έχει πολλές προγραμματισμένες βάρδιες και κάθε συγκεκριμένη βάρδια ανήκει ακριβώς σε ένα τμήμα. Η σχέση είναι 1:N και παρουσιάζει ολική συμμετοχή στην πλευρά των departments.

- **Shifts – Personnel**

Ένα μέλος προσωπικού συμμετέχει σε πολλές βάρδιες κατά τη διάρκεια του μήνα και κάθε βάρδια στελεχώνεται από πολλά μέλη του προσωπικού. Η σχέση είναι M:N και παρουσιάζει ολική συμμετοχή στη πλευρά της βάρδιας, καθώς κάθε βάρδια πρέπει να στελεχώνεται από προσωπικό.

- **Departments – Rooms**

Κάθε τμήμα περιέχει πολλές κλίνες και κάθε κλίνη υπάγεται σε ακριβώς ένα τμήμα. Η σχέση είναι 1:N και παρουσιάζει ολική συμμετοχή στις κλίνες, καθώς κάθε κλίνη πρέπει να ανήκει σε ένα τμήμα.

- **Admission – Various Relations**

Ο πίνακας admission αποτελεί τον κεντρικό κόμβο της βάσης δεδομένων και συνδέεται με τις περισσότερες οντότητες του νοσοκομείου. Οι εν λόγω σχέσεις είναι οι εξής:

- 1. Admission – Rooms**

Κάθε νοσηλεία πραγματοποιείται σε μία κλίνη και κάθε κλίνη μπορεί να φιλοξενήσει διάφορες νοσηλείες. Η σχέση είναι N:1 με ολική συμμετοχή στη πλευρά της νοσηλείας.

- 2. Admission – Departments**

Κάθε νοσηλεία πραγματοποιείται σε ένα τμήμα και κάθε τμήμα φιλοξενεί διάφορες νοσηλείες. Η σχέση είναι N:1 με ολική συμμετοχή στη πλευρά της νοσηλείας.

- 3. Admission – Diagnosis**

Κάθε νοσηλεία περιλαμβάνει μία διάγνωση εισαγωγής και μία διάγνωση εξόδου, επομένως υφίσταται μία διπλή σχέση τύπου 1:N με ολική συμμετοχή στη πλευρά της νοσηλείας.

- 4. Admission – Ken**

Κάθε νοσηλεία αντιστοιχίζεται σε έναν μοναδικό κωδικό KEN και κάθε κωδικός KEN μπορεί να αντιστοιχιστεί σε διάφορες νοσηλείες. Η σχέση είναι N:1 με ολική συμμετοχή στη πλευρά της νοσηλείας.

- 5. Admission - Patients**

Ένας ασθενής μπορεί να έχει πραγματοποιήσει πολλές νοσηλείες στο νοσοκομείο και μία νοσηλεία αφορά έναν ακριβώς ασθενή. Η σχέση είναι N:1 με ολική συμμετοχή στην πλευρά των νοσηλειών.

- 6. Admission – Emergency Case**

Ένας ασθενής πρώτα καταφθάνει στα επείγοντα (γίνεται εγγραφή στον πίνακα Emergency Case) και στη συνέχεια ενδεχομένως πραγματοποιείται η εισαγωγή του και η νοσηλεία του στο νοσοκομείο. Κάθε νοσηλεία υποχρεωτικά έχει προέλθει από ένα περιστατικό στα επείγοντα. Επομένως, η σχέση είναι 1:1 με ολική συμμετοχή στη πλευρά της νοσηλείας.

7. Admission – Medical Acts

Μία νοσηλεία μπορεί να περιλαμβάνει μία ή περισσότερες ιατρικές πράξεις (πχ χειρουργείο) και κάθε ιατρική πράξη αντιστοιχίζεται σε ακριβώς μία νοσηλεία. Η σχέση είναι 1:N με ολική συμμετοχή στη πλευρά των ιατρικών πράξεων.

8. Admission – Lab Exams

Η σχέση είναι όμοια με την Admission – Medical Acts.

9. Admission – Prescription

Η σχέση είναι όμοια με την Admission – Medical Acts.

10. Admission – Admission Evaluation

Ο ασθενής ενδεχομένως αξιολογεί την εμπειρία του για την νοσηλεία και η σχέση είναι 1:1 με ολική συμμετοχή στη πλευρά της αξιολόγησης.

11. Admission – Doctor Evaluation

Ο ασθενής ενδεχομένως αξιολογεί κάθε γιατρό που του συνταγογράφησε κάποιο φάρμακο στα πλαίσια της νοσηλείας του. Η σχέση είναι 1:N με ολική συμμετοχή στη πλευρά του Doctor Evaluation.

- **Medical Acts – Various Relations**

1. Medical Acts – Medical Act Codes

Κάθε ιατρική πράξη αντιστοιχίζεται σε έναν μοναδικό κωδικό. Η σχέση είναι N:1 και παρουσιάζει ολική συμμετοχή στη πλευρά των ιατρικών πράξεων.

2. Medical Acts – Doctors

Κάθε ιατρική πράξη πραγματοποιείται από έναν κύριο χειρουργό και ένας χειρουργός μπορεί να εκτελεί διάφορες ιατρικές πράξεις. Η σχέση είναι N:1 με ολική συμμετοχή στη πλευρά της πράξης.

3. Medical Acts – Personnel

Μία ιατρική πράξη ενδέχεται να στελεχώνεται και από μία ομάδα βοηθών (ιατρών ή νοσηλευτών) που είναι μέλη του προσωπικού. Η σχέση είναι M:N και δεν παρουσιάζει ολική συμμετοχή.

4. Medical Acts – KEN

Κάθε ιατρική πράξη έχει ένα κόστος, το οποίο περιγράφεται από τον αντίστοιχο κωδικό ΚΕΝ. Επομένως, η σχέση μεταξύ τους είναι N:1 και παρουσιάζεται ολική συμμετοχή στη πλευρά της ιατρικής πράξης.

5. Medical Acts – Rooms

Κάθε ιατρική πράξη λαμβάνει χώρα σε μία κλίνη και κάθε κλίνη φιλοξενεί διάφορες ιατρικές πράξεις κατά τη λειτουργία του νοσοκομείου. Η σχέση είναι N:1 και παρουσιάζει ολική συμμετοχή στη πλευρά της ιατρικής πράξης.

- **Lab – Exams – Various Relations**

1. Lab Exams – Doctors

Κάθε εργαστηριακή εξέταση πραγματοποιείται από έναν ιατρό και ένας ιατρός μπορεί να έχει εκτελέσει διάφορες εξετάσεις στην καριέρα του. Η σχέση είναι N:1 και παρουσιάζει ολική συμμετοχή στη πλευρά των εξετάσεων.

2. Lab Exams – Lab Exam Codes

Κάθε εργαστηριακή εξέταση αντιστοιχίζεται σε έναν μοναδικό κωδικό. Η σχέση είναι N:1 και παρουσιάζει ολική συμμετοχή στη πλευρά των εξετάσεων.

3. Lab Exams – KEN

Κάθε εργαστηριακή εξέταση έχει ένα κόστος, το οποίο περιγράφεται από τον αντίστοιχο κωδικό ΚΕΝ. Επομένως, η σχέση μεταξύ τους είναι N:1 και παρουσιάζεται ολική συμμετοχή στη πλευρά της εξέτασης.

- **Prescription – Various Relations**

1. Prescription – Patients

Κάθε συνταγογράφηση αφορά έναν ασθενή και ένας ασθενής μπορεί να διαθέτει διάφορες συνταγογραφήσεις. Η σχέση είναι N:1 και παρουσιάζει ολική συμμετοχή στη πλευρά της συνταγογράφησης.

2. Prescription – Medicine

Κάθε συνταγογράφηση περιλαμβάνει ένα φάρμακο και ένα φάρμακο μπορεί να συνταγογραφηθεί πολλές φορές. Η σχέση είναι N:1 και παρουσιάζει ολική συμμετοχή στη πλευρά της συνταγογράφησης. Το κάθε φάρμακο περιλαμβάνει διάφορες δραστικές ουσίες οι οποίες βρίσκονται στο weak entity Medicine Substances. Η σχέση είναι M:N, καθώς οι δραστικές ουσίες μπορούν να περιέχονται σε πολλά φάρμακα, ενώ η ολική συμμετοχή είναι αμφίδρομη, καθώς δεν μπορείς να έχεις φάρμακο δίχως δραστική ουσία και το αντίστροφο.

3. Prescription – Doctors

Κάθε συνταγογράφηση έχει εκτελεστεί από έναν ιατρό και ένας ιατρός συνταγογραφεί πολλαπλές φορές. Η σχέση είναι N:1 και παρουσιάζει ολική συμμετοχή στη πλευρά της συνταγογράφησης.

- **Emergency Case – Various Relations**

- 1. Emergency Case – Patients**

Κάθε επείγον περιστατικό προφανώς αφορά έναν ασθενή και ένας ασθενής μπορεί να καταφθάσει πολλαπλές φορές στα επείγοντα. Η σχέση είναι N:1 και παρουσιάζει ολική συμμετοχή στη πλευρά των επειγόντων.

- 2. Emergency Case – Nurses**

Κάθε ασθενής που καταφθάνει στα επείγοντα, εξετάζεται από έναν νοσηλευτή διαλογής και ένας νοσηλευτής μπορεί να εξετάσει πολλά περιστατικά. Η σχέση είναι N:1 και παρουσιάζει ολική συμμετοχή στη πλευρά των επειγόντων περιστατικών.

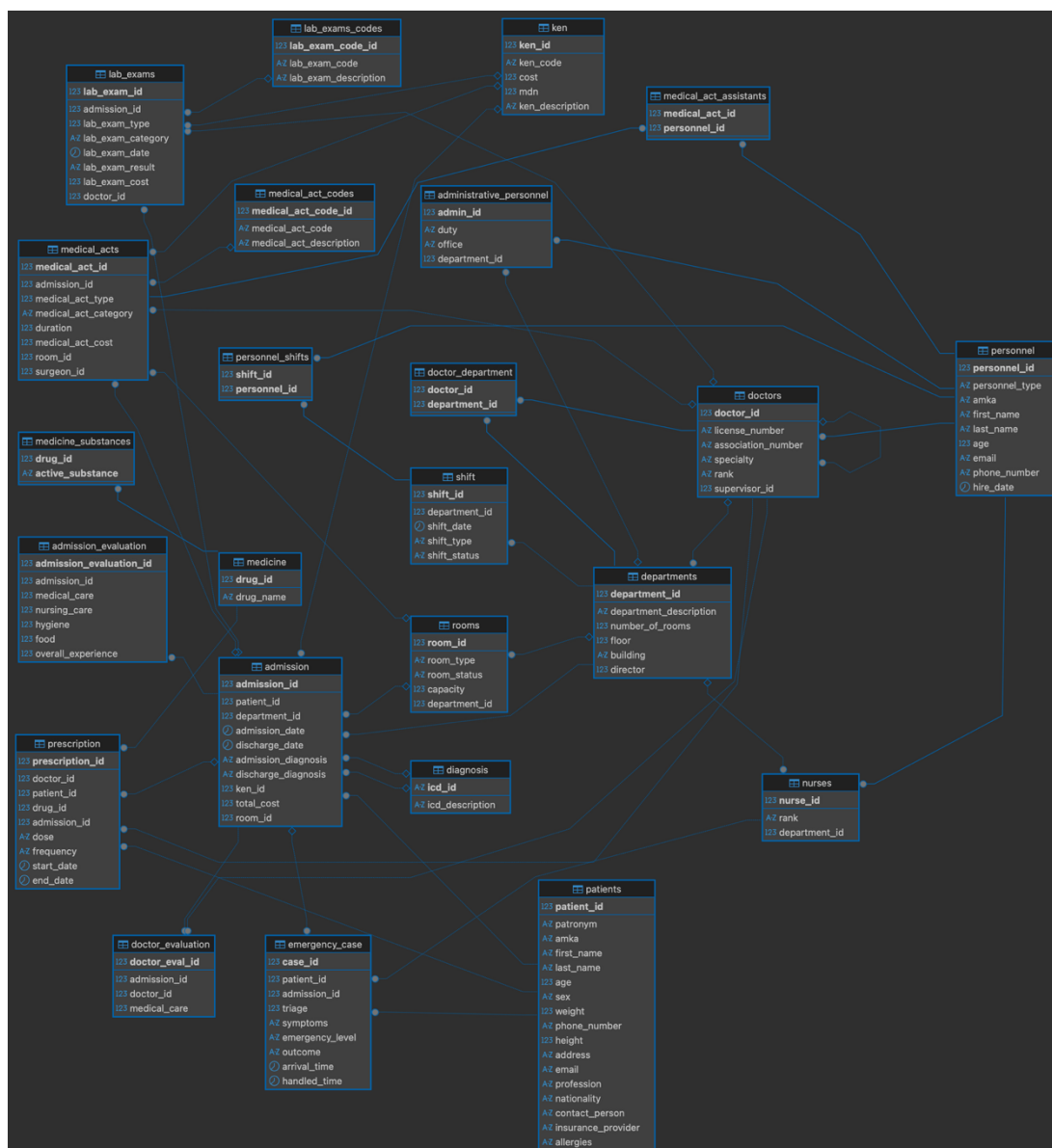
- **Doctor Evaluation – Doctors**

Κάθε αξιολόγηση ιατρού, προφανώς αφορά έναν ιατρό και κάθε ιατρός μπορεί να αξιολογηθεί πολλές φορές από τους ασθενείς του, καθώς συνταγογραφεί πολλαπλές φορές. Η σχέση είναι N:1 και παρουσιάζει ολική συμμετοχή στη πλευρά της αξιολόγησης.

A.2 Υλοποίηση Βάσης Δεδομένων

A.2.1 Σχεσιακό Διάγραμμα

Κατά την διαδικασία μετατροπής του ER μας σε Σχεσιακό Διάγραμμα, έπρεπε να διαχειριστούμε κατάλληλα τις σχέσεις μεταξύ των οντοτήτων. Συγκεκριμένα, οι **Σχέσεις 1:1** αποδίδονται μεταφέροντας το Primary Key του ενός πίνακα ως Foreign Key στον άλλον. Η επιλογή του πίνακα του οποίου το κύριο κλειδί θα μπει ως ξένο κλειδί στον άλλον δεν είναι αυστηρά καθορισμένη. Οι **Σχέσεις 1:N** εκφράζονται με τον ίδιο τρόπο, με την προϋπόθεση πως το Primary Key βρίσκεται στην πλευρά του «Ένα» (1) και το Foreign Key τοποθετείται στην πλευρά των «Πολλών» (N). Από την άλλη, οι **Σχέσεις M:N** απαιτούν την δημιουργία ενός ενδιάμεσου πίνακα. Ο πίνακας αυτός εξασφαλίζει ότι κάθε συσχέτιση είναι μοναδική συνδυάζοντας τα δύο κλειδιά των αρχικών πινάκων σε ένα νέο σύνθετο Primary Key για τον ίδιο. Τα κλειδιά αυτά είναι παράλληλα, το καθένα ξεχωριστά, Foreign Key του νέου πίνακα.



A.2.2 Περιορισμοί Ορθότητας

Constraints

Προχωρήσαμε σε ορισμό κατάλληλων περιορισμών, που εξασφαλίζουν τη λογική ορθότητα της βάσης μας, όπως αυτοί ζητήθηκαν από την εκφώνηση.

- Μοναδικότητα Συνταγογράφησης.

```
ALTER TABLE prescription
ADD CONSTRAINT unique_prescription_combo
UNIQUE (doctor_id, patient_id, drug_id, start_date);
```

Ο συνδυασμός ιατρού, ασθενούς και φαρμάκου, ορίζεται ως μοναδικός για κάθε μεμονωμένη συνταγογράφηση. Στην υλοποίησή μας, η ημερομηνία έναρξης ορίζεται ως η αμέσως επόμενη από την ημέρα ολοκλήρωσης της νοσηλείας. Επομένως προστέθηκε στον περιορισμό για να εξασφαλίσουμε ότι ο παραπάνω συνδυασμός πρέπει να είναι μοναδικός στο πλαίσιο μιας συγκεκριμένης νοσηλείας. Με τον τρόπο αυτόν αποτρέπονται οι διπλές καταχωρήσεις, ενώ παράλληλα επιτρέπεται η εκ νέου χορήγηση της ίδιας αγωγής στον ίδιο ασθενή σε μελλοντική νοσηλεία.

- Μοναδικότητα Βάρδιας

```
ALTER TABLE shift
ADD CONSTRAINT unique_department_date_shift
UNIQUE (department_id, shift_date, shift_type);
```

Για τον πίνακα βαρδιών, ορίστηκε περιορισμός μοναδικότητας στον συνδυασμό του τμήματος, της ημερομηνίας και του τύπου της βάρδιας. Τύπος βάρδιας στη βάση μας σημαίνει πρωινή, απογευματινή ή βραδινή. Η επιλογή αυτή εξασφαλίζει ότι υπάρχει μία και μοναδική βάρδια την ίδια μέρα και ώρα σε ένα συγκεκριμένο τμήμα.

Triggers

Για τους πιο σύνθετους επιχειρησιακούς κανόνες του νοσοκομείου, οι οποίοι δεν μπορούν να καλυφθούν από τους απλούς περιορισμούς του σχήματος, αναπτύξαμε κατάλληλα Triggers. Αυτά εκτελούνται αυτόματα κατά την τροποποίηση των δεδομένων, προστατεύοντας ενεργά τη λογική συνοχή του συστήματος. Οι SQL κώδικες των παρακάτω υπάρχουν στο αρχείο 'triggers.sql'.

- validate_supervisor:

Αυτό το trigger ενεργοποιείται πριν από κάθε εισαγωγή ή ενημέρωση του πίνακα 'doctors' και διασφαλίζει την εποπτική ιεραρχία του νοσοκομείου. Αρχικά, ελέγχει τους βασικούς κανόνες της δομής, επιβάλλοντας ότι ένας Διευθυντής δεν μπορεί να έχει επόπτη, ενώ αντιθέτως ένας Ειδικευόμενος επιβάλλεται να εποπτεύεται από άλλον Ιατρό. Επιπλέον η συνάρτηση ελέγχει τους ιεραρχικούς κανόνες όπως αυτοί ορίζονται στην εκφώνηση. Κάθε επόπτης πρέπει να έχει βαθμίδα μεγαλύτερη από αυτή του εποπτευόμενου.

Συγκεκριμένα, ο επόπτης ενός Ειδικευόμενου πρέπει να είναι τουλάχιστον Επιμελητής Β', ο επόπτης ενός Επιμελητή Β' πρέπει να τουλάχιστον Επιμελητής Α', ενώ ο επόπτης ενός Επιμελητή Α' είναι αναγκαστικά Διευθυντής.

- individual_shift_constraints:

Το συγκεκριμένο trigger ενεργοποιείται πριν από κάθε εισαγωγή εγγραφής στον πίνακα 'personnel_shifts' και είναι υπεύθυνο για την τήρηση των εργασιακών περιορισμών κάθε υπαλλήλου. Αρχικά, αποτρέπει την ανάθεση της ίδιας βάρδιας σε έναν εργαζόμενο περισσότερες από μία φορές, αποφεύγοντας τις διπλές εγγραφές στη βάση. Επιβάλλει το μηνιαίο όριο βαρδιών ανάλογα με την ειδικότητα του εργαζομένου, δηλαδή επιτρέπει έως 15 βάρδιες σε έναν Ιατρό, 20 σε έναν Νοσηλευτή και 25 σε ένα μέλος Διοικητικού Προσωπικού. Επιπλέον ελέγχει την υποχρεωτική 8ωρη ανάπαυση, απαγορεύοντας την εργασία σε δύο συνεχόμενες βάρδιες. Τέλος, διασφαλίζει ότι κανένας υπάλληλος του νοσοκομείου δεν εργάζεται σε 3 συνεχόμενες νυχτερινές βάρδιες.

- validate_shift:

Αυτό το trigger ενεργοποιείται πριν από κάθε ενημέρωση του status μιας βάρδιας στον πίνακα 'shift' και συγκεκριμένα όταν η βάρδια γίνεται 'Completed'. Σε πρώτο στάδιο, επιβεβαιώνει ότι έχουν ανατεθεί οι ελάχιστοι απαιτούμενοι εργαζόμενοι ανά τμήμα, δηλαδή αν η βάρδια διαθέτει τουλάχιστον 3 Ιατρούς, 6 Νοσηλευτές και 2 μέλη Διοικητικού Προσωπικού. Παράλληλα, διασφαλίζει πως αν συμμετέχει κάποιος Ειδικευόμενος Ιατρός σε μία βάρδια, τότε υπάρχει απαραίτητα στο ίδιο ωράριο και ένας Επιμελητής Α' ή Διευθυντής για την επίβλεψή του.

- check_room_capacity:

Το trigger αυτό ενεργοποιείται πριν από κάθε νέα εισαγωγή εγγραφής στον πίνακα admission και λειτουργεί ως μηχανισμός ελέγχου για την αποτροπή υπεράριθμων νοσηλείων σε ένα δωμάτιο. Αρχικά, υπολογίζει τον τρέχοντα αριθμό ασθενών που νοσηλεύονται στον συγκεκριμένο θάλαμο, καταμετρώντας τις εγγραφές που δεν έχουν καταχωρημένη ημερομηνία εξιτηρίου. Έπειτα, ανακτά τη μέγιστη επιτρεπτή χωρητικότητα του δωματίου και ελέγχει αν ο αριθμός των ήδη νοσηλευόμενων ασθενών ισούται ή υπερβαίνει τη χωρητικότητα αυτή. Σε τέτοια περίπτωση, η διαδικασία της εισαγωγής διακόπτεται.

- update_ward_status:

Το συγκεκριμένο trigger εκτελείται μετά από κάθε εισαγωγή νοσηλείας ή ενημέρωση της ημερομηνίας εξιτηρίου. Σκοπός του είναι η έγκαιρη ενημέρωση της διαθεσιμότητας κάθε θαλάμου. Συγκεκριμένα, μετράει τους ασθενείς του δωματίου και συγκρίνει το πλήθος τους με τη δηλωμένη χωρητικότητα. Αν ο θάλαμος έχει γεμίσει, ενημερώνει την κατάσταση σε 'Κατειλημμένο'. Αντιθέτως, όταν έχουμε αναχώρηση κάποιου ασθενούς, ενημερώνει το status του δωματίου στο οποίο νοσηλευόταν, σε 'Διαθέσιμο'.

- check_allergy_before_prescription:

Αυτό το trigger ενεργοποιείται πριν από κάθε εισαγωγή νέας συνταγής ή ενημέρωση υπάρχουσας στον πίνακα 'prescription', αποτρέποντας τη συνταγογράφηση φαρμάκων που περιέχουν δραστικές ουσίες, στις οποίες ο εκάστοτε ασθενής είναι αλλεργικός. Η συνάρτηση ανακτά την καταγεγραμμένη αλλεργία του ασθενούς και ελέγχει για τυχόν ταύτιση με τα συστατικά του υπό συνταγογράφηση φαρμάκου. Σε τέτοια περίπτωση, αποτρέπει την εγγραφή στον πίνακα.

A.2.3 Συναρτήσεις και Διαδικασίες

Για την ολοκληρωμένη λειτουργία και την αρχικοποίηση της βάσης μας, αναπτύχθηκε ένα σύνολο από συναρτήσεις και διαδικασίες. Ο ρόλος τους είναι αφενός να εξασφαλίζουν την εισαγωγή νέων δεδομένων στο σύστημα και αφετέρου να αυτοματοποιούν την μαζική παραγωγή ρεαλιστικών δεδομένων συνδυάζοντας ήδη υπάρχουσες εγγραφές. Κάποιες από αυτές μάλιστα επιβάλλουν κανόνες και περιορισμούς του νοσοκομείου, πριν αυτοί ελεγχθούν σε δεύτερο βαθμό από τα triggers. Οι SQL κώδικες των παρακάτω υπάρχουν στο αρχείο 'insert_functions.sql'.

Functions

- insert_doctor_function :

Φτιάξαμε αυτή τη συνάρτηση για να κάνουμε πιο γρήγορη και ασφαλή την προσθήκη ιατρών στη βάση μας, καθώς η χειροκίνητη εισαγωγή ήταν αρκετά περίπλοκη λόγω των συσχετίσεων. Ουσιαστικά, η συνάρτηση παίρνει όλα τα στοιχεία ενός ιατρού και, πριν τον αποθηκεύσει, φροντίζει να του αναθέσει αυτόματα έναν τυχαίο αλλά ιεραρχικά σωστό επόπτη με βάση τη βαθμίδα του ώστε να μην χτυπάνε οι περιορισμοί μας. Στη συνέχεια, αναλαμβάνει να μοιράσει τα δεδομένα σωστά: περνάει τα προσωπικά στοιχεία στον πίνακα 'personnel', τα ιατρικά στον πίνακα 'doctors' και τέλος τρέχει μια λούπα για να συνδέσει κατευθείαν τον ιατρό με τις κλινικές στις οποίες εργάζεται στον ενδιαμέσο πίνακα 'doctor_department'. Με αυτόν τον τρόπο, γλιτώσαμε τα πολλαπλά χειροκίνητα INSERTS και διασφάλισαμε ότι τα δεδομένα των ιατρών μας μπήκαν με απόλυτη συνοχή.

- insert_admin_function :

Αντίστοιχα με τους ιατρούς, δημιουργήσαμε αυτή τη συνάρτηση για να διευκολύνουμε τη μαζική εισαγωγή εγγραφών για το Διοικητικό Προσωπικό. Επειδή η δομή μας απαιτεί τα γενικά στοιχεία να μπαίνουν στον πίνακα personnel και τα ειδικά στον πίνακα 'administrative_personnel', η συνάρτηση αυτοματοποιεί αυτή τη διαδικασία με ένα μόνο βήμα. Πρώτα περνάει τα στοιχεία στον γενικό πίνακα και κρατάει το νέο ID που δημιουργήθηκε. Το πιο σημαντικό όμως, είναι ότι για να εξασφαλίσουμε μια ρεαλιστική κατανομή των υπαλλήλων σε όλο το νοσοκομείο, η συνάρτηση διαλέγει αυτόματα και τυχαία ένα τμήμα από τον πίνακα departments, στο οποίο θα υπάγεται ο υπάλληλος, και

στη συνέχεια αποθηκεύει τον υπάλληλο, μαζί με τα καθήκοντα και το γραφείο του, στον ειδικό πίνακα του διοικητικού προσωπικού.

- insert_nurse_function :

Επειδή και οι νοσηλευτές κληρονομούν τα χαρακτηριστικά τους από τον κεντρικό πίνακα του προσωπικού, η συνάρτηση αναλαμβάνει να κάνει την εξής δουλειά για εμάς: δημιουργεί πρώτα την εγγραφή στον πίνακα 'personnel', δηλώνοντας αυτόματα τον τύπο ως 'Νοσηλευτής', και κρατάει το νέο ID που δημιουργήθηκε. Αμέσως μετά, για να αποφύγουμε την αυθαίρετη ή χειροκίνητη τοποθέτησή τους, το σύστημα επιλέγει εντελώς τυχαία ένα τμήμα από τον πίνακα departments. Στο τελικό βήμα, παίρνει το ID, το τυχαίο τμήμα και τη βαθμίδα του νοσηλευτή και τα καταχωρεί στον ειδικό πίνακα 'nurses'.

Procedures

- generate_random_rooms

Η συνάρτηση αυτή είναι υπεύθυνη για τη δημιουργία δωματίων και την καταχώρησή τους στον πίνακα 'rooms'. Αρχικά, ανατρέχει σε όλα τα καταχωρημένα τμήματα και με βάση τον δηλωμένο αριθμό δωματίων τους, δημιουργεί τις αντίστοιχες εγγραφές. Κατά τη δημιουργία αναθέτει τυχαία την κατάσταση, καθώς και τον τύπο του δωματίου. Η κατάσταση αρχικοποιείται ως 'Διαθέσιμο', εκτός από λίγες εξαιρέσεις που ορίζονται ως 'Υπό Συντήρηση'. Με βάση το αν ο τύπος είναι 'Μονόκλινο', 'Δίκλινο' ή κάτι άλλο, ορίζεται και η χωρητικότητα.

- process_emergency_cases:

Αυτή η διαδικασία διαχειρίζεται τα επείγοντα περιστατικά του νοσοκομείου. Αρχικά, εξυπηρετεί τα ανοιχτά περιστατικά, σύμφωνα με τις προτεραιότητες που ορίζει η εκκώφηση. Για τα περιστατικά εκείνα που απαιτούν νοσηλεία, δημιουργεί την αντίστοιχη εγγραφή στον πίνακα 'admission', αναθέτοντας τυχαία ένα τμήμα, δωμάτιο, κωδικό KEN από τον πίνακα 'ken' και διάρκεια. Αντιστοιχεί επίσης τις διαγνώσεις παίρνοντας έναν κωδικό 'ICD-10' από τον πίνακα 'diagnosis'.

- populate_lab_exams:

Η διαδικασία αυτή δημιουργεί 600 εργαστηριακές εξετάσεις για τον πίνακα 'lab_exams'. Λειτουργεί επιλέγοντας τυχαίες νοσηλείες και αναθέτοντάς τους έναν τυχαίο αριθμό εξετάσεων, από 1 έως 4. Για κάθε εξέταση επιλέγεται αν η κατηγορία της θα είναι 'Αιματολογική', 'Βιοχημική' ή 'Απεικονιστική'. Ορίζεται επίσης ο ιατρός που την παρήγγειλε, ένας κωδικός KEN που θα αντιπροσωπεύει το κόστος, καθώς και ο τύπος της εξέτασης. Τύπο εξέτασης θεωρούμε ένα τυχαίο αναγνωριστικό του πίνακα

‘lab_exam_codes’. Τέλος, παράγεται ένα ρεαλιστικό αποτέλεσμα για την εκάστοτε εξέταση, το οποίο εξαρτάται από την κατηγορία της.

- populate_medical_acts:

Αυτή η διαδικασία δημιουργεί 600 εγγραφές ιατρικών πράξεων και επεμβάσεων. Οι πράξεις αυτές συνδέονται με ήδη υπάρχουσες νοσηλείες και εισάγονται στον πίνακα ‘medical_acts’. Αυτές κατανέμονται ως ‘Χειρουργικές’, ‘Διαγνωστικές’ ή ‘Θεραπευτικές’. Ορίζεται τυχαία ο χώρος εκτέλεσης, καθώς και ο υπεύθυνος ιατρός. Σε περίπτωση χειρουργικής πράξης, αυτή ανατίθεται σε χειρουργό και λαμβάνει χώρα αποκλειστικά σε αίθουσα ΜΕΘ ή χειρουργείο. Επιλέγεται επίσης τυχαία μια διάρκεια, εντός των ορίων της συνολικής διάρκειας νοσηλείας. Τέλος, η διαδικασία αναθέτει τυχαία έναν αριθμό βοηθών από το διαθέσιμο προσωπικό του νοσοκομείου και εισάγει τα απαραίτητα αναγνωριστικά στον πίνακα ‘medical_act_assistants’.

- populate_prescriptions:

Η διαδικασία αυτή είναι υπεύθυνη για την εισαγωγή στον πίνακα ‘prescription’. Επιλέγει τυχαία 300 νοσηλείες και για την κάθε μία δημιουργεί συνταγή 2 έως 4 φαρμάκων. Πραγματοποιείται έλεγχος για ουσίες των φαρμάκων στις οποίες ο ασθενής μπορεί να είναι αλλεργικός. Δεν προκρίνεται ποτέ μια τέτοια συνταγογράφηση. Για κάθε έγκυρο φάρμακο επιλέγεται ένας συνταγογράφων ιατρός, μία τυπική δοσολογία, καθώς και η συχνότητα λήψης. Τέλος, η ημερομηνία έναρξης της αγωγής, ορίζεται ως η επόμενη μέρα από την ημερομηνία εξιτηρίου του ασθενούς, με διάρκεια που κυμαίνεται από 3 έως 15 ημέρες.

- calculate_total_admission_cost:

Αυτή η διαδικασία εκτελεί την κοστολόγηση όλων των νοσηλείων του νοσοκομείου, ενημερώνοντας το πεδίο ‘total_cost’ του πίνακα ‘admission’. Το συνολικό κόστος κάθε νοσηλείας υπολογίζεται με βάση τρεις επιμέρους παράγοντες. Πρώτον, υπολογίζεται το βασικό κόστος νοσηλείας, σύμφωνα με το ΚΕΝ που έχει ανατεθεί σε αυτήν. Στο ποσό αυτό προστίθεται μία αναλογική ημερήσια χρέωση σε περίπτωση που η συνολική διάρκεια παραμονής του ασθενούς ξεπερνάει τη Μέση Διάρκεια Νοσηλείας του συγκεκριμένου ΚΕΝ. Δεύτερον, υπολογίζεται και προστίθεται το άθροισμα του κόστους όλων των ιατρικών πράξεων που πραγματοποιήθηκαν κατά τη διάρκεια της νοσηλείας. Τρίτον, στο τελικό ποσό αθροίζεται το συνολικό κόστος όλων των εργαστηριακών εξετάσεων στις οποίες υποβλήθηκε ο ασθενής.

- populate_evaluations:

Η διαδικασία αυτή αυτοματοποιεί τη δημιουργία και καταχώρηση δεδομένων αξιολόγησης που αφορούν ολοκληρωμένες νοσηλείες. Σε πρώτο στάδιο, το σύστημα ανακτά τα μοναδικά ζεύγη ιατρού – νοσηλείας από το ιστορικό συνταγογραφήσεων και καταχωρεί μία τυχαία βαθμολογία, στην κλίμακα 1 έως 5, στο πεδίο ‘medical_care’ του

πίνακα 'doctor_evaluation'. Στη συνέχεια, για την αξιολόγηση των γενικότερων συνθηκών της νοσηλείας, παράγονται τυχαίες βαθμολογίες για τα επιμέρους κριτήρια. Αυτές εισάγονται στα αντίστοιχα πεδία του πίνακα 'admission_evaluation'. Τέλος, υπολογίζεται το 'overall_experience' ως ο μέσος όρος των παραπάνω.

A.3 Mock Data

Για την τροφοδότηση της βάσης με τον απαραίτητο, μεγάλο όγκο ρεαλιστικών δεδομένων, υιοθετήθηκε μια μικτή στρατηγική, προσαρμόζοντας το εργαλείο παραγωγής στις εκάστοτε απαιτήσεις της κάθε οντότητας. Συγκεκριμένα, ακολουθήθηκαν δύο βασικές προσεγγίσεις:

1. Σενάρια Python σε συνδυασμό με Συναρτήσεις SQL

Για το ιατρικό, νοσηλευτικό και διοικητικό προσωπικό, αναπτύχθηκαν εξειδικευμένα scripts σε Python, αξιοποιώντας και εργαλεία τεχνητής νοημοσύνης. Κάνοντας χρήση της βιβλιοθήκης Faker, παρήχθησαν αληθοφανή στοιχεία, όπως ΑΜΚΑ, τηλέφωνα, ονοματεπώνυμα, ενσωματώνοντας ταυτόχρονα αυστηρούς προγραμματιστικούς περιορισμούς. Για παράδειγμα, στους ιατρούς εφαρμόστηκε αυστηρή αντιστοίχιση ειδικοτήτων-τμημάτων και ποσοστιαία κατανομή ιεραρχίας, στους νοσηλευτές σταθμισμένες πιθανότητες βαθμίδων, και στο διοικητικό προσωπικό ειδικός αλγόριθμος αρίθμησης γραφείων. Τα scripts αυτά δεν έκαναν απευθείας εισαγωγές στη βάση, αλλά παρήγαγαν έτοιμα αρχεία SQL, τα οποία με τη σειρά τους καλούσαν τις συναρτήσεις εισαγωγής, όπως τις είδαμε παραπάνω.

2. Σενάρια Python με παραγωγή καθαρής SQL

Για οντότητες με αυστηρούς χρονικούς, συνδυαστικούς ή στατιστικούς περιορισμούς, προτιμήθηκε η απευθείας παραγωγή εντολών 'INSERT INTO' μέσω Python.

- **Ασθενείς:** Για τη δημιουργία του μητρώου ασθενών, το script δεν περιορίστηκε στα βασικά δημογραφικά στοιχεία. Εφαρμόστηκε εξειδικευμένη στατιστική λογική χρησιμοποιώντας σταθμισμένες πιθανότητες για το ιατρικό τους προφίλ. Έτσι, διασφαλίστηκε ότι η συντριπτική πλειοψηφία των ασθενών δεν έχει γνωστές αλλεργίες, ενώ οι υπόλοιποι κατανέμονται με ρεαλιστικά ιατρικά ποσοστά σε γνωστές φαρμακευτικές ουσίες. Τα δεδομένα μορφοποιήθηκαν σε ένα ενιαίο, ασφαλές αρχείο πολλαπλών INSERT.
- **Βάρδιες Προσωπικού:** Το script διάβαζε τα παραγόμενα CSVs του προσωπικού και έλυνε ένα πρόβλημα χρονοπρογραμματισμού. Επέβαλε τους κανόνες που ορίζονται στην εκφώνηση, όπως σεβασμός του 8ωρου ανάπαυσης, αποτροπή 3 συνεχόμενων νυχτερινών βαρδιών, μηνιαία όρια και κανόνες ιεραρχίας.

- **Επείγοντα Περιστατικά:** Προσομοιώθηκε ένα πλήρες χρονολόγιο διαιτίας, 2025-2026. Ο αλγόριθμος υπολόγιζε ρεαλιστικά κενά μεταξύ των επισκέψεων του ίδιου ασθενούς και σταθμισμένα επίπεδα σοβαρότητας

3. Μαζική Εισαγωγή Δεδομένων Αναφοράς

Τέλος, για τα πραγματικά ιατρικά δεδομένα που μας δόθηκαν μέσω εξωτερικών συνδέσμων (διαγνώσεις ICD-10, KEN, φάρμακα, εργαστηριακές εξετάσεις και ιατρικές πράξεις), ακολουθήθηκε μια διαφορετική προσέγγιση. Αρχικά, τα αρχεία Excel υποβλήθηκαν στην απαραίτητη προεπεξεργασία και έπειτα εξήχθησαν σε μορφή CSV, με την κατάλληλη κωδικοποίηση (UTF-8) ώστε να διασφαλιστεί η σωστή αναπαράσταση των ελληνικών χαρακτήρων. Η τελική φόρτωση στη βάση υλοποιήθηκε μέσω του εργαλείου Data Import του DBeaver. Μέσω του γραφικού περιβάλλοντος, πραγματοποιήθηκαν μαζικές εισαγωγές, με εφαρμογή του κατάλληλου column mapping, ώστε οι στήλες των CSV να αντιστοιχιστούν με απόλυτη ακρίβεια στα πεδία των αντίστοιχων πινάκων του σχήματός μας.

B. Queries

B1. Βρείτε τα συνολικά έσοδα του νοσοκομείου ανά τμήμα και ανά έτος, με ανάλυση ανά KEN κωδικό (βασικό κόστος έναντι πρόσθετης χρέωσης λόγω υπέρβασης ΜΔΝ) και κατανομή νοσηλειών ανά ασφαλιστικό φορέα.

```
SELECT
    d.department_description AS "Τμήμα",
    EXTRACT(YEAR FROM a.admission_date) AS "Έτος",
    p.insurance_provider AS "Ασφαλ. Φορέας",
    COUNT(a.admission_id) AS "Νοσηλείες",
    SUM(a.total_cost) AS "Συνολικά Έσοδα",
    SUM(k.cost) AS "Βασικό Κόστος",
    SUM(
        GREATEST(0, (COALESCE(a.discharge_date, CURRENT_DATE) - a.admission_date) - k.mdn)
        * (k.cost / NULLIF(k.mdn, 0))
    ) AS "Χρέωση Υπέρβασης ΜΔΝ"
FROM
    admission a
JOIN
    departments d ON a.department_id = d.department_id
JOIN
    patients p ON a.patient_id = p.patient_id
JOIN
    ken k ON a.ken_id = k.ken_id
GROUP BY
    d.department_description,
    EXTRACT(YEAR FROM a.admission_date),
    p.insurance_provider
ORDER BY
    "Έτος" DESC,
    "Τμήμα",
    "Συνολικά Έσοδα" DESC;
```


Το παραπάνω query υπολογίζει τα ετήσια έσοδα κάθε τμήματος κατηγοριοποιημένα ανά έτος και ανά ασφαλιστικό φορέα ασθενών. Μας επιστρέφονται μέσω SELECT τα τμήματα, τα διαφορετικά έτη, οι ασφαλιστικοί φορείς των ασθενών, το πλήθος των νοσηλείων, το άθροισμα των εσόδων από κάθε νοσηλεία, καθώς και οι πρόσθετες χρεώσεις που επιβάλλονται στους ασθενείς λόγω υπέρβασης της Μέσης Διάρκειας Νοσηλείας. Τα παραπάνω δεδομένα εξάγονται από τους πίνακες 'admission', 'departments', 'patients' και 'ken' οι οποίοι συνενώνονται με χρήση JOINS. Για την εύρεση του κόστους λόγω υπέρβασης ΜΔΝ αφαιρούμε τις ημερομηνίες εξιτηρίου και εισαγωγής για να υπολογίσουμε την συνολική διάρκεια νοσηλείας. Αν δεν υπάρχει ημερομηνία εξιτηρίου καταχωρημένη στο σύστημα, η συνάρτηση COALESCE αναθέτει ως τέτοια την σημερινή ημερομηνία. Αν η διάρκεια νοσηλείας υπερβαίνει την ΜΔΝ, τότε οι μέρες υπέρβασης πολλαπλασιάζονται με την ημερήσια επιβάρυνση. Για την παραγωγή των αθροισμάτων SUM και τον υπολογισμό του COUNT, απαραίτητη είναι η ομαδοποίηση μέσω GROUP BY των δεδομένων, βάσει του τμήματος, του έτους εισαγωγής και του ασφαλιστικού φορέα. Τα αποτελέσματα ανά τμήμα παρουσιάζονται για φθίνουσα τιμή έτους και συνολικών εσόδων.

B2: Για συγκεκριμένη ειδικότητα ιατρού, βρείτε όλους τους ιατρούς που ανήκουν σε αυτήν, με ένδειξη αν είχαν εφημερία το τρέχον έτος και πόσες επεμβάσεις εκτέλεσαν ως κύριοι χειρουργοί.

```
WITH ShiftsThisYear AS (
    SELECT DISTINCT ps.personnel_id AS doctor_id
    FROM personnel_shifts ps
    JOIN shift s ON ps.shift_id = s.shift_id
    WHERE EXTRACT(YEAR FROM s.shift_date) = EXTRACT(YEAR FROM CURRENT_DATE)
),
SurgeriesAsLead AS (
    SELECT
        surgeon_id AS doctor_id,
        COUNT(*) AS total_surgeries
    FROM medical_acts
    WHERE surgeon_id IS NOT NULL
    GROUP BY surgeon_id
)
SELECT
    d.doctor_id AS "ID Ιατρού",
    p.first_name AS "Όνομα",
    p.last_name AS "Επώνυμο",
    d.specialty AS "Ειδικότητα",

    CASE
        WHEN sty.doctor_id IS NOT NULL THEN 'NAI'
        ELSE 'OXI'
    END AS "Εφημερία Φέτος",

    COALESCE(sal.total_surgeries, 0) AS "Σύνολο Επεμβάσεων"

FROM doctors d
JOIN personnel p ON d.doctor_id = p.personnel_id
LEFT JOIN ShiftsThisYear sty ON d.doctor_id = sty.doctor_id
LEFT JOIN SurgeriesAsLead sal ON d.doctor_id = sal.doctor_id
WHERE d.specialty = 'Χειρουργός';
```

Το παραπάνω query ανακτά τα στοιχεία των ιατρών μιας συγκεκριμένης ειδικότητας (στην περίπτωσή μας «Χειρουργός»), αξιολογώντας παράλληλα τη συμμετοχή τους σε εφημερίες κατά το τρέχον έτος και τον συνολικό αριθμό επεμβάσεων που έχουν πραγματοποιήσει ως κύριοι χειρουργοί. Μας επιστρέφονται μέσω της SELECT το αναγνωριστικό του ιατρού, το όνομα, το επώνυμο, η ειδικότητά του, μια λεκτική ένδειξη (NAI/OXI) για το αν εκτέλεσε εφημερία φέτος, καθώς και το πλήθος των επεμβάσεων. Για την αποφυγή λανθασμένων πολλαπλασιασμών των εγγραφών λόγω του ότι ένας ιατρός μπορεί να έχει πολλαπλές βάρδιες και πολλαπλές επεμβάσεις, το ερώτημα δομείται με χρήση Common Table Expressions (WITH). Η προσωρινή δομή ShiftsThisYear φιλτράρει τους ιατρούς που έχουν εφημερία το τρέχον έτος με τη βοήθεια της συνάρτησης EXTRACT(YEAR), ενώ η δομή SurgeriesAsLead ομαδοποιεί και υπολογίζει το πλήθος των επεμβάσεων ανά ιατρό (COUNT). Στη συνέχεια, τα κύρια δεδομένα εξάγονται από τους πίνακες doctors και personnel με χρήση JOIN και ενώνονται με τα παραπάνω CTEs μέσω LEFT JOIN. Το LEFT JOIN είναι απαραίτητο ώστε να μην αποκλειστούν από τα αποτελέσματα οι χειρουργοί που δεν έχουν καταγεγραμμένες επεμβάσεις ή εφημερίες. Τέλος, χρησιμοποιείται η συνθήκη CASE για τη μετατροπή της ύπαρξης εγγραφής εφημερίας στη λεκτική ένδειξη «NAI» ή «OXI», και η συνάρτηση COALESCE η οποία φροντίζει να αναθέσει την τιμή 0 στον μετρητή των επεμβάσεων στις περιπτώσεις που ο ιατρός επιστρέφει NULL. Το φιλτράρισμα της επιθυμητής ειδικότητας υλοποιείται μέσω της εντολής WHERE.

B3. Βρείτε ποιοι ασθενείς έχουν νοσηλευτεί περισσότερες από 3 φορές στο ίδιο τμήμα, με το συνολικό κόστος νοσηλείας τους.

```
SELECT
    p.patient_id,
    p.first_name,
    p.last_name,
    d.department_description,
    COUNT(a.admission_id) AS total_admissions,
    SUM(a.total_cost) AS total_cost_in_department
FROM admission a
JOIN patients p ON a.patient_id = p.patient_id
JOIN departments d ON a.department_id = d.department_id
GROUP BY
    p.patient_id,
    p.first_name,
    p.last_name,
    d.department_id,
    d.department_description

HAVING COUNT(a.admission_id) > 3
ORDER BY total_admissions DESC;
```

Το συγκεκριμένο ερώτημα εντοπίζει ασθενείς με επαναλαμβανόμενες εισαγωγές (περισσότερες από 3) στο ίδιο τμήμα του νοσοκομείου, υπολογίζοντας παράλληλα το

συνολικό κόστος αυτών των νοσηλειών. Μας επιστρέφονται μέσω της SELECT το αναγνωριστικό του ασθενούς, το όνομα, το επώνυμο, η περιγραφή του τμήματος, το συνολικό πλήθος των νοσηλειών του στο συγκεκριμένο τμήμα, καθώς και το άθροισμα του κόστους όλων αυτών των νοσηλειών. Τα απαραίτητα δεδομένα αντλούνται από τον κεντρικό πίνακα νοσηλειών admission, ο οποίος συνενώνεται με τον πίνακα των ασθενών patients και τον πίνακα των τμημάτων departments με τη χρήση JOINS. Για την εξαγωγή των συγκεντρωτικών αποτελεσμάτων, χρησιμοποιείτε η εντολή GROUP BY. Η ομαδοποίηση πραγματοποιείται με βάση τα στοιχεία του ασθενούς (patient_id, first_name, last_name) και τα στοιχεία του τμήματος (department_id, department_description), ώστε οι υπολογισμοί να γίνονται ανά συνδυασμό ασθενούς-τμήματος. Για την καταμέτρηση των νοσηλειών χρησιμοποιείται η συνάρτηση COUNT(a.admission_id) και για τον υπολογισμό του συνολικού κόστους η συνάρτηση SUM(a.total_cost). Το φίλτράρισμα των ασθενών που έχουν νοσηλευτεί περισσότερες από τρεις φορές στο ίδιο τμήμα υλοποιείται με την εντολή HAVING COUNT(a.admission_id) > 3, η οποία εφαρμόζεται μετά την ομαδοποίηση. Τέλος, τα αποτελέσματα ταξινομούνται φθίνουσα (ORDER BY total_admissions DESC) με βάση τον αριθμό των εισαγωγών.

B4. Για συγκεκριμένο ιατρό, βρείτε τον μέσο όρο αξιολογήσεων των ασθενών του (κριτήριο: Ποιότητα ιατρικής φροντίδας) και τη Συνολική εντύπωση νοσηλείας.

```
EXPLAIN ANALYZE
SELECT
  d.doctor_id AS "Κωδικός Ιατρού",
  p.first_name AS "Όνομα",
  p.last_name AS "Επώνυμο",
  ROUND(AVG(de.medical_care), 2) AS "Μέση Αξιολόγηση Ιατρού (Φροντίδα)",
  ROUND(AVG(ae.overall_experience), 2) AS "Μέση Αξιολόγηση Νοσηλείας (Συνολική)"
FROM doctors d
JOIN personnel p ON d.doctor_id = p.personnel_id
JOIN doctor_evaluation de ON d.doctor_id = de.doctor_id
JOIN admission_evaluation ae ON de.admission_id = ae.admission_id
WHERE d.doctor_id = 11
GROUP BY
  d.doctor_id,
  p.first_name,
  p.last_name;
```

Το παραπάνω query υπολογίζει τον μέσο όρο των αξιολογήσεων που έχει λάβει ένας συγκεκριμένος ιατρός (με αναγνωριστικό d.doctor_id = 11), τόσο όσον αφορά την ποιότητα της ιατρικής φροντίδας όσο και τη συνολική εμπειρία νοσηλείας των ασθενών του. Μας επιστρέφονται μέσω της SELECT ο κωδικός του ιατρού, το όνομα, το επώνυμο και οι δύο μέσοι όροι. Τα δεδομένα αντλούνται από τον πίνακα doctors, ο οποίος συνενώνεται με τον πίνακα personnel (για την ανάκτηση των προσωπικών στοιχείων), καθώς και με τους πίνακες αξιολογήσεων doctor_evaluation και admission_evaluation. Η συνένωση του τελευταίου γίνεται μέσω του admission_id, προκειμένου να αντιστοιχηθεί η αξιολόγηση του ιατρού με την αξιολόγηση της εκάστοτε νοσηλείας. Για τον υπολογισμό των μέσων όρων χρησιμοποιείται η συνάρτηση AVG, ενώ η ROUND στρογγυλοποιεί τα

αποτελέσματα σε δύο δεκαδικά ψηφία για βέλτιστη αναγνωσιμότητα. Επιπλέον, γίνεται η χρήση της εντολής GROUP BY στα στοιχεία του ιατρού, ώστε να παραχθούν τα συγκεντρωτικά αποτελέσματα. Το φιλτράρισμα για τον συγκεκριμένο ιατρό υλοποιείται με την εντολή WHERE.

(α) Σε πρώτη φάση, το ερώτημα εκτελέστηκε με χρήση της εντολής EXPLAIN ANALYZE, προκειμένου να αναλυθεί το σχέδιο εκτέλεσης στο περιβάλλον της PostgreSQL, χωρίς την παρουσία κάποιου εξειδικευμένου ευρετηρίου.

```
QUERY PLAN
GroupAggregate (cost=49.59..49.64 rows=1 width=102) (actual time=0.278..0.280 rows=0 loops=1)
  Group Key: d.doctor_id, p.first_name, p.last_name
  -> Sort (cost=49.59..49.60 rows=2 width=46) (actual time=0.276..0.278 rows=0 loops=1)
    Sort Key: p.first_name, p.last_name
    Sort Method: quicksort  Memory: 25kB
    -> Nested Loop (cost=0.83..49.58 rows=2 width=46) (actual time=0.256..0.257 rows=0 loops=1)
      -> Nested Loop (cost=0.55..32.98 rows=2 width=46) (actual time=0.255..0.256 rows=0 loops=1)
        -> Nested Loop (cost=0.55..16.60 rows=1 width=38) (actual time=0.032..0.036 rows=1 loops=1)
          -> Index Only Scan using doctors_pkey on doctors d (cost=0.28..8.29 rows=1 width=4) (actual time=0.016..0.017 rows=1 loops=1)
            Index Cond: (doctor_id = 11)
            Heap Fetches: 1
          -> Index Scan using personnel_pkey on personnel p (cost=0.28..8.29 rows=1 width=38) (actual time=0.011..0.012 rows=1 loops=1)
            Index Cond: (personnel_id = 11)
        -> Seq Scan on doctor_evaluation de (cost=0.00..16.36 rows=2 width=12) (actual time=0.218..0.218 rows=0 loops=1)
          Filter: (doctor_id = 11)
          Rows Removed by Filter: 909
      -> Index Scan using admission_evaluation_admission_id_key on admission_evaluation ae (cost=0.28..8.29 rows=1 width=8) (never executed)
        Index Cond: (admission_id = de.admission_id)
Planning Time: 0.608 ms
Execution Time: 0.408 ms
```

Η βάση εφάρμοσε Σειριακή Σάρωση (Seq Scan on doctor_evaluation de), διαβάζοντας σειριακά τις εγγραφές για να εντοπίσει αυτές που αντιστοιχούν στον ιατρό με ID 11. Όπως παρατηρούμε στο πλάνο, η βάση αναγκάστηκε να διαβάσει και να απορρίψει 909 άσχετες εγγραφές (Rows Removed by Filter: 909). Το συνολικό εκτιμώμενο κόστος ανήλθε στο 49.64, ενώ ο πραγματικός χρόνος εκτέλεσης στα 0.408 ms.

(β) Στο σημείο αυτό, δημιουργούμε ένα ευρετήριο, το οποίο εφαρμόζεται στο attribute doctor_id του πίνακα doctor_evaluation. Επειδή η PostgreSQL δεν υποστηρίζει άμεσα την εντολή FORCE INDEX, προκειμένου να εξαναγκάσουμε τον query planner να χρησιμοποιήσει το ευρετήριο, απενεργοποιούμε προσωρινά τη σειριακή σάρωση μέσω του planner hint SET enable_seqscan = OFF;.

```
CREATE INDEX idx_q4_doctor_eval ON doctor_evaluation(doctor_id);
```

```
SET enable_seqscan = OFF;
```

Και στο τέλος προστίθεται και το :

```
SET enable_seqscan = ON;
```

Πλέον παίρνουμε την εξής έξοδο :

```

QUERY PLAN
-----
GroupAggregate (cost=41.75..41.80 rows=1 width=102) (actual time=0.063..0.064 rows=0 loops=1)
  Group Key: d.doctor_id, p.first_name, p.last_name
  -> Sort (cost=41.75..41.75 rows=2 width=46) (actual time=0.062..0.062 rows=0 loops=1)
    Sort Key: p.first_name, p.last_name
    Sort Method: quicksort Memory: 25kB
    -> Nested Loop (cost=5.12..41.74 rows=2 width=46) (actual time=0.056..0.057 rows=0 loops=1)
      -> Nested Loop (cost=4.84..25.14 rows=2 width=46) (actual time=0.056..0.057 rows=0 loops=1)
        -> Nested Loop (cost=0.55..16.60 rows=1 width=38) (actual time=0.027..0.029 rows=1 loops=1)
          -> Index Only Scan using doctors_pkey on doctors d (cost=0.28..8.29 rows=1 width=4) (actual time=0.011..0.012 rows=1 loops=1)
            Index Cond: (doctor_id = 11)
            Heap Fetches: 1
          -> Index Scan using personnel_pkey on personnel p (cost=0.28..8.29 rows=1 width=38) (actual time=0.012..0.012 rows=1 loops=1)
            Index Cond: (personnel_id = 11)
        -> Bitmap Heap Scan on doctor_evaluation de (cost=4.29..8.52 rows=2 width=12) (actual time=0.026..0.026 rows=0 loops=1)
          Recheck Cond: (doctor_id = 11)
          -> Bitmap Index Scan on idx_q4_doctor_eval (cost=0.00..4.29 rows=2 width=0) (actual time=0.024..0.024 rows=0 loops=1)
            Index Cond: (doctor_id = 11)
      -> Index Scan using admission_evaluation_admission_id_key on admission_evaluation ae (cost=0.28..8.29 rows=1 width=8) (never executed)
        Index Cond: (admission_id = de.admission_id)
Planning Time: 0.564 ms
Execution Time: 0.112 ms

```

Η βάση χρησιμοποίησε το ευρετήριο (Bitmap Index Scan on idx_q4_doctor_eval). Το συνολικό εκτιμώμενο κόστος αυτή τη φορά μειώθηκε στο 41.80 και ο χρόνος εκτέλεσης έπεσε στα 0.112 ms.

(γ) Συγκρίνοντας τα δύο πλάνα εκτέλεσης, παρατηρούμε ότι η λύση με χρήση ευρετηρίου είχε μικρότερο εκτιμώμενο κόστος, 41.80 έναντι 49.64 της Σειριακής Σάρωσης. Επιπλέον, ο χρόνος εκτέλεσης μειώθηκε δραστικά, από 0.408 ms σε 0.112 ms (σχεδόν 4 φορές ταχύτερο).

(δ) Σύμφωνα με τα αποτελέσματα, είναι σαφές πως το σύστημα εξοικονομεί σημαντικούς υπολογιστικούς πόρους όταν εγκαταλείπει την πλήρη σάρωση του πίνακα και επιλέγει να χρησιμοποιήσει το κατάλληλο ευρετήριο. Το ευρετήριο επέτρεψε στη βάση να εντοπίσει απευθείας τις εγγραφές του συγκεκριμένου ιατρού, αποφεύγοντας το φιλτράρισμα των 909 περιττών γραμμών, αποδεικνύοντας έτσι την αποδοτικότητά του ακόμα και σε πίνακες με σχετικά μικρό όγκο δεδομένων.

B5. Βρείτε τους νέους ιατρούς (ηλικία < 35 ετών) που έχουν εκτελέσει τις περισσότερες χειρουργικές επεμβάσεις ως κύριοι χειρουργοί.

```

SELECT
  p.first_name AS "Όνομα",
  p.last_name AS "Επώνυμο",
  d.specialty AS "Ειδικότητα",
  p.age AS "Ηλικία",
  COUNT(ma.medical_act_id) AS "Σύνολο Χειρουργείων"
FROM
  personnel p
JOIN
  doctors d ON p.personnel_id = d.doctor_id
JOIN
  medical_acts ma ON d.doctor_id = ma.surgeon_id
WHERE
  p.age < 35
  AND ma.medical_act_category = 'Χειρουργική'
GROUP BY
  d.doctor_id,
  p.first_name,
  p.last_name,
  d.specialty,
  p.age
ORDER BY
  "Σύνολο Χειρουργείων" DESC;

```

Διαλέγουμε μέσω SELECT όνομα και επώνυμο του γιατρού, την ειδικότητά του, την ηλικία του, καθώς και τον συνολικό αριθμό χειρουργείων. Τα παραπάνω θα αντληθούν

από τους πίνακες 'personnel', 'doctors' και 'medical_acts' με χρήση JOINS. Συγκεκριμένα, ο πίνακας του προσωπικού που γίνεται joined με αυτόν των γιατρών, επιλέγει αναγνωριστικά 'personnel_id' όταν αυτά τα βρίσκει και ως 'doctor_id' στον αντίστοιχο πίνακα. Με χρήση του WHERE, θα επιλεχθούν γιατροί κάτω των 35 ετών και αυστηρά Χειρουργικές Ιατρικές Πράξεις. Τα δεδομένα ομαδοποιούνται, με GROUP BY, ανά ιατρό, ώστε η συνάρτηση COUNT να υπολογίσει το σύνολο των επεμβάσεων για τον καθένα ξεχωριστά. Τα αποτελέσματα παρουσιάζονται σε φθίνουσα σειρά ως πριν τον αριθμό χειρουργείων.

B6. Για συγκεκριμένο ασθενή, βρείτε το ιστορικό νοσηλειών του, τις αντίστοιχες διαγνώσεις (ICD-10), το συνολικό κόστος ανά νοσηλεία και τον μέσο όρο αξιολόγησής του.

```
SELECT
  a.admission_id AS "Κωδικός Νοσηλείας",
  a.admission_date AS "Ημ/νία Εισαγωγής",
  a.discharge_date AS "Ημ/νία Εξιτηρίου",
  din.icd_description AS "Διάγνωση Εισαγωγής",
  dout.icd_description AS "Διάγνωση Εξιτηρίου",
  a.total_cost AS "Συνολικό Κόστος",
  ROUND((ae.medical_care + ae.nursing_care + ae.hygiene + ae.food + ae.overall_experience) / 5.0, 2) AS "Μέση Αξιολόγηση"
FROM
  admission a
LEFT JOIN
  diagnosis din ON a.admission_diagnosis = din.icd_id
LEFT JOIN
  diagnosis dout ON a.discharge_diagnosis = dout.icd_id
LEFT JOIN
  admission_evaluation ae ON a.admission_id = ae.admission_id
WHERE
  a.patient_id = 5;
```

Το παραπάνω query βρίσκει το πλήρες ιστορικό νοσηλειών του ασθενή με id 5. Εκτελεί 3 LEFT JOINS του πίνακα 'admission' με τους πίνακες 'diagnosis' και 'admission_evaluation'. Κατά την διαδικασία των LEFT JOINS, επιλέγει αναγνωριστικά διαγνώσεων εισαγωγής και εξόδου από τον πίνακα 'diagnosis' όταν αυτά ταυτίζονται με τις διαγνώσεις νοσηλειών. Αντίστοιχα επιλέγονται οι αξιολογήσεις που αντιστοιχούν σε υπάρχουσα νοσηλεία. Η χρήση των LEFT JOINS αντί INNER JOINS, επιλέχτηκε διότι το query πρέπει να κρατάει όλες τις νοσηλείες, ακόμα και αν για αυτές δεν έχει βρεθεί διάγνωση εξιτηρίου ή αξιολόγηση, δηλαδή αν ο ασθενής είναι ακόμα στο νοσοκομείο. Τα παραπάνω δεδομένα φιλτράρονται και κρατάμε μόνο το ιστορικό του ασθενούς με id 5. Η SELECT επιλέγει τον κωδικό της νοσηλείας, ημερομηνίες εισαγωγής και εξιτηρίου, διάγνωση εισαγωγής και εξιτηρίου, συνολικό κόστος νοσηλείας, καθώς και την μέση αξιολόγηση. Αυτή υπολογίζεται ως μέσος όρος των επιμέρους κατηγοριών. Με τη βοήθεια της ROUND, στρογγυλοποιούμε το αποτέλεσμα στα δύο δεκαδικά ψηφία.

(α) Σε πρώτη φάση, το ερώτημα εκτελέστηκε με χρήση της εντολής EXPLAIN ANALYZE, προκειμένου να αναλυθεί το σχέδιο εκτέλεσης. Τρέξαμε τον κώδικα τοπικά σε PostgreSQL περιβάλλον.


```

QUERY PLAN
-----
Nested Loop Left Join (cost=32.09..102.02 rows=3 width=220) (actual time=0.357..0.627 rows=3 loops=1)
  -> Nested Loop Left Join (cost=31.81..77.06 rows=3 width=127) (actual time=0.329..0.564 rows=3 loops=1)
    -> Hash Right Join (cost=31.53..52.15 rows=3 width=46) (actual time=0.303..0.499 rows=3 loops=1)
      Hash Cond: (ae.admission_id = a.admission_id)
      -> Seq Scan on admission_evaluation ae (cost=0.00..17.99 rows=999 width=24) (actual time=0.007..0.150 rows=999 loops=1)
      -> Hash (cost=31.49..31.49 rows=3 width=26) (actual time=0.206..0.207 rows=3 loops=1)
        Buckets: 1024 Batches: 1 Memory Usage: 9kB
        -> Seq Scan on admission a (cost=0.00..31.49 rows=3 width=26) (actual time=0.075..0.186 rows=3 loops=1)
          Filter: (patient_id = 5)
          Rows Removed by Filter: 996
    -> Index Scan using diagnosis_pkey on diagnosis din (cost=0.29..8.30 rows=1 width=91) (actual time=0.019..0.019 rows=1 loops=3)
      Index Cond: (icd_id = a.admission_diagnosis)
  -> Index Scan using diagnosis_pkey on diagnosis dout (cost=0.29..8.30 rows=1 width=91) (actual time=0.017..0.017 rows=1 loops=3)
    Index Cond: (icd_id = a.discharge_diagnosis)
Planning Time: 0.828 ms
Execution Time: 0.681 ms

```

Η βάση εφάρμοσε Σειριακή Σάρωση, καθώς όπως παρατηρούμε αναφέρεται η χρήση 'Seq Scan on admission a'. Διαβάστηκε κάθε εγγραφή του πίνακα admission, και λόγω της απαίτησης 'patient_id = 5' το σύστημα αναγκάστηκε να απορρίψει τις υπόλοιπες 996 εγγραφές. Στο πλάνο αυτό, το συνολικό εκτιμώμενο κόστος ανήλθε στο 102.02, ενώ ο πραγματικός χρόνος εκτέλεσης στα 0.681 ms.

(β) Στο σημείο αυτό δημιουργούμε ένα ευρετήριο, το οποίο εφαρμόζεται στο attribute 'patient_id' του πίνακα 'admission'. Πριν τρέξουμε το query μαζί με το EXPLAIN ANALYZE, οφείλουμε να απενεργοποιήσουμε την σειριακή σάρωση.

```
CREATE INDEX idx_q6_patient ON admission(patient_id);
```

```
SET enable_seqscan = OFF;
```

Παίρνουμε την εξής έξοδο:

```

QUERY PLAN
-----
Nested Loop Left Join (cost=5.14..87.52 rows=3 width=220) (actual time=0.078..0.118 rows=3 loops=1)
  -> Nested Loop Left Join (cost=4.87..62.58 rows=3 width=188) (actual time=0.061..0.096 rows=3 loops=1)
    -> Nested Loop Left Join (cost=4.58..37.67 rows=3 width=107) (actual time=0.050..0.070 rows=3 loops=1)
      -> Bitmap Heap Scan on admission a (cost=4.30..12.76 rows=3 width=26) (actual time=0.035..0.036 rows=3 loops=1)
        Recheck Cond: (patient_id = 5)
        Heap Blocks: exact=2
      -> Bitmap Index Scan on idx_q6_patient (cost=0.00..4.30 rows=3 width=0) (actual time=0.032..0.032 rows=3 loops=1)
        Index Cond: (patient_id = 5)
    -> Index Scan using diagnosis_pkey on diagnosis din (cost=0.29..8.30 rows=1 width=91) (actual time=0.009..0.009 rows=1 loops=3)
      Index Cond: (icd_id = a.admission_diagnosis)
  -> Index Scan using diagnosis_pkey on diagnosis dout (cost=0.29..8.30 rows=1 width=91) (actual time=0.008..0.008 rows=1 loops=3)
    Index Cond: (icd_id = a.discharge_diagnosis)
  -> Index Scan using admission_evaluation_admission_id_key on admission_evaluation ae (cost=0.28..8.29 rows=1 width=24) (actual time=0.005..0.005 rows=1 loops=3)
    Index Cond: (admission_id = a.admission_id)
Planning Time: 0.583 ms
Execution Time: 0.149 ms

```

Η βάση πράγματι χρησιμοποίησε το ευρετήριο 'Bitmap Index Scan on idx_q6_patient'. Το συνολικό εκτιμώμενο κόστος αυτή τη φορά ήταν 87.52 και ο χρόνος εκτέλεσης 0.149 ms.

(γ) Παρατηρούμε ότι η λύση με χρήση ευρετηρίου είχε μικρότερο εκτιμώμενο κόστος, 87.52 έναντι 102.02 της Σειριακής Σάρωσης. Δηλαδή, το σύστημα καταναλώνει ελαφρώς λιγότερους υπολογιστικούς πόρους όταν διαθέτει ένα INDEX. Επιπλέον ο χρόνος εκτέλεσης χρήσης INDEX είναι σχεδόν 5 φορές μικρότερος, 0.149 ms έναντι 0.681 ms.

(δ) Σύμφωνα με τα αποτελέσματα που παρουσιάστηκαν παραπάνω, είναι σαφές πως το σύστημα εξοικονομεί υπολογιστικούς πόρους και παρουσιάζει αισθητά βελτιωμένο χρόνο εκτέλεσης, όταν εγκαταλείπει την Σειριακή Σάρωση και επιλέγει να χρησιμοποιήσει κατάλληλο ευρετήριο. Συνεπώς, η χρήση INDEX σε προβλήματα παρόμοιας φύσης κρίνεται ιδιαίτερα αποδοτική.

B7. Βρείτε για κάθε δραστική ουσία τον αριθμό ασθενών που έχουν δηλώσει αλλεργία και τον αριθμό φαρμάκων που την περιέχουν, ταξινομημένα κατά συνολικό αριθμό αλλεργικών ασθενών.

```
SELECT
|   ms.active_substance AS "Δραστική Ουσία",
|   COUNT(DISTINCT ms.drug_id) AS "Αριθμός Φαρμάκων",
|   COUNT(DISTINCT p.patient_id) AS "Αριθμός Αλλεργικών Ασθενών"
FROM
|   medicine_substances ms
JOIN
|   patients p ON p.allergies ILIKE ms.active_substance
WHERE
|   p.allergies IS NOT NULL
GROUP BY
|   ms.active_substance
ORDER BY
|   "Αριθμός Αλλεργικών Ασθενών" DESC;
```

Ο παραπάνω κώδικας υπολογίζει το πλήθος των ασθενών που είναι αλλεργικοί σε κάθε δραστική ουσία και τον συνολικό αριθμό των φαρμάκων που την περιέχουν. Χρησιμοποιούμε συνένωση με JOIN των πινάκων 'patients' και 'medicine_substances', κρατώντας μόνο τις δραστικές ουσίες στις οποίες κάποιος ασθενής έχει δηλώσει αλλεργία. Η συνθήκη αυτή ελέγχεται με τη βοήθεια της εντολής ON, ενώ ο τελεστής ILIKE εξασφαλίζει ότι η αναζήτηση δεν είναι ευαίσθητη σε πεζά ή κεφαλαία γράμματα. Η εντολή WHERE απορρίπτει τους ασθενείς χωρίς δηλωμένη αλλεργία. Τα δεδομένα ομαδοποιούνται ανά δραστική ουσία. Το query επιλέγει τις δραστικές ουσίες, τον αριθμό των φαρμάκων που τις περιέχουν, καθώς και τον αριθμό των αλλεργικών ασθενών σε αυτές. Φροντίζουμε μέσω του DISTINCT, τα τελικά μας αποτελέσματα να μην έχουν διπλομετρήσει κάποιον ασθενή ή κάποιο φάρμακο. Ένα τέτοιο σφάλμα θα μπορούσε να γίνει διότι μία δραστική ουσία μπορεί να περιέχεται σε παραπάνω από ένα φάρμακο αλλά και αντίστροφα, πολλοί διαφορετικοί ασθενείς μπορεί να είναι αλλεργικοί στην ίδια ουσία. Η παρουσίαση των αποτελεσμάτων γίνεται κατά φθίνοντα αριθμό αλλεργικών ασθενών.

B8. Βρείτε το προσωπικό (ιατροί, νοσηλευτές, διοικητικό προσωπικό) που δεν έχει προγραμματισμένη εφημερία σε συγκεκριμένη ημερομηνία και τμήμα.

```
SELECT
|   p.personnel_id,
|   p.first_name,
|   p.last_name,
|   p.personnel_type
FROM
|   personnel p
WHERE
|   p.personnel_id NOT IN (
|       SELECT
|       |   ps.personnel_id
|       FROM
|       |   personnel_shifts ps
|       JOIN
|       |   shift s ON ps.shift_id = s.shift_id
|       WHERE
|       |   s.shift_date = '2026-05-15'
|       |   AND s.department_id = 1
|   )
ORDER BY
|   p.personnel_type,
|   p.last_name;
```


Το ερώτημα ζητάει το προσωπικό το οποίο δεν έχει προγραμματισμένη βάρδια για συγκεκριμένη ημερομηνία σε συγκεκριμένο τμήμα. Ο τρόπος που επιλέξαμε να προσεγγίσουμε το ερώτημα είναι η τεχνική του subquery σε συνδυασμό με τον τελεστή NOT IN. Το εμφωλευμένο ερώτημα συνενώνει με χρήση JOIN τους πίνακες 'personnel_shifts' και 'shift' και κρατάει μέσω του WHERE τις βάρδιες που πραγματοποιήθηκαν την 15/05/2026 στο τμήμα 1. Το προσωπικό που επιλέγεται με την SELECT εξαιρείται με χρήση του τελεστή NOT IN από το κύριο query. Το query αυτό εκτελεί SELECT τα στοιχεία όλου του υπόλοιπου προσωπικού και τα ταξινομεί στην έξοδο πρωτίστως με βάση την ειδικότητα και δευτερευόντως με το επώνυμο.

B9. Βρείτε ποιοι ασθενείς νοσηλεύτηκαν τον ίδιο αριθμό ημερών σε διάστημα ενός έτους, με συνολική διάρκεια άνω των 15 ημερών.

```
WITH YearlyPatientStats AS (  
    SELECT  
        p.patient_id,  
        p.first_name,  
        p.last_name,  
        EXTRACT(YEAR FROM a.admission_date) AS admission_year,  
        SUM(COALESCE(a.discharge_date, CURRENT_DATE) - a.admission_date) AS total_days  
    FROM admission a  
    JOIN patients p ON a.patient_id = p.patient_id  
    WHERE  
        EXTRACT(YEAR FROM a.admission_date) = 2025  
    GROUP BY  
        p.patient_id,  
        p.first_name,  
        p.last_name,  
        EXTRACT(YEAR FROM a.admission_date)  
    HAVING  
        SUM(COALESCE(a.discharge_date, CURRENT_DATE) - a.admission_date) > 15  
)  
GroupedPatients AS (  
    SELECT  
        patient_id,  
        first_name AS "Όνομα",  
        last_name AS "Επώνυμο",  
        admission_year AS "Έτος",  
        total_days AS "Συνολικές Ημέρες",  
        COUNT(patient_id) OVER (PARTITION BY total_days) AS patients_with_same_days  
    FROM YearlyPatientStats  
)  
SELECT  
    "Όνομα",  
    "Επώνυμο",  
    "Έτος",  
    "Συνολικές Ημέρες"  
FROM GroupedPatients  
WHERE patients_with_same_days > 1  
ORDER BY "Συνολικές Ημέρες" DESC, "Επώνυμο";
```

Το συγκεκριμένο ερώτημα εντοπίζει ασθενείς που έχουν συνολική διάρκεια νοσηλείας άνω των 15 ημερών κατά τη διάρκεια ενός επιλεγμένου έτους (στο παράδειγμά μας, το 2025) και επιστρέφει αποκλειστικά εκείνους που τυχαίνει να μοιράζονται ακριβώς τον ίδιο αριθμό ημερών νοσηλείας με τουλάχιστον έναν ακόμη ασθενή. Το query δομείται σε λογικά στάδια κάνοντας χρήση Common Table Expressions (WITH). Η πρώτη προσωρινή δομή, YearlyPatientStats, αναλαμβάνει να συνενώσει τον πίνακα των νοσηλειών admission με τον πίνακα patients (μέσω JOIN) και να υπολογίσει το σύνολο των ημερών

νοσηλείας ανά ασθενή. Η χρονική διάρκεια κάθε νοσηλείας προκύπτει από την αφαίρεση της ημερομηνίας εισαγωγής από την ημερομηνία εξιτηρίου. Η χρήση της συνάρτησης COALESCE διασφαλίζει ότι, αν ένας ασθενής δεν έχει λάβει ακόμη εξιτήριο (NULL), ο υπολογισμός των ημερών γίνεται δυναμικά με βάση τη σημερινή ημερομηνία (CURRENT_DATE). Τα δεδομένα ομαδοποιούνται ανά ασθενή (GROUP BY) και διατηρούνται μόνο όσοι ξεπερνούν τις 15 ημέρες συνολικά μέσω της εντολής HAVING. Παράλληλα, χρησιμοποιείται η συνάρτηση EXTRACT(YEAR) προκειμένου να απομονωθούν οι εγγραφές του έτους 2025. Η δεύτερη δομή GroupedPatients αντλεί τα δεδομένα της προηγούμενης και αξιοποιεί μια αναλυτική συνάρτηση, την COUNT(patient_id) OVER (PARTITION BY total_days). Η συνάρτηση αυτή μετράει πόσοι ασθενείς έχουν ακριβώς τον ίδιο αριθμό συνολικών ημερών (total_days), χωρίς ωστόσο να συμπτύσσει τις γραμμές, επιτρέποντας τη διατήρηση των προσωπικών στοιχείων κάθε ασθενούς. Τέλος, το SELECT επιλέγει τα επιθυμητά πεδία (Όνομα, Επώνυμο, Έτος, Συνολικές Ημέρες) και εφαρμόζει το τελικό φίλτρο WHERE patients_with_same_days > 1. Αυτό εγγυάται ότι θα εμφανιστούν αποκλειστικά οι ασθενείς για τους οποίους υπάρχει «σύμπτωση» ημερών. Τα τελικά αποτελέσματα ταξινομούνται φθίνουσα ως προς τις συνολικές ημέρες και αλφαβητικά ως προς το επώνυμο για βέλτιστη αναγνωσιμότητα.

B10. Πολλοί ασθενείς λαμβάνουν συνδυασμούς φαρμάκων. Βρείτε τα top-3 ζεύγη δραστικών ουσιών που συνταγογραφήθηκαν ταυτόχρονα στον ίδιο ασθενή κατά την ίδια νοσηλεία, ταξινομημένα κατά συχνότητα εμφάνισης.

```
WITH AdmissionSubstances AS (
    SELECT DISTINCT
        p.admission_id,
        ms.active_substance
    FROM prescription p
    JOIN medicine_substances ms ON p.drug_id = ms.drug_id
    WHERE p.admission_id IS NOT NULL
)
SELECT
    a1.active_substance AS "Δραστική Ουσία 1",
    a2.active_substance AS "Δραστική Ουσία 2",
    COUNT(*) AS Συχνότητα
FROM AdmissionSubstances a1
JOIN AdmissionSubstances a2
    ON a1.admission_id = a2.admission_id
    AND a1.active_substance < a2.active_substance
GROUP BY
    a1.active_substance,
    a2.active_substance
ORDER BY
    Συχνότητα DESC
LIMIT 3;
```

Αρχικά, δημιουργούμε έναν προσωρινό πίνακα (CTE) με το όνομα AdmissionSubstances ώστε να αντιστοιχηθεί κάθε νοσηλεία με τις δραστικές ουσίες των φαρμάκων που

χορηγήθηκαν. Στη συνέχεια, κάνουμε self-join τον προσωρινό πίνακα και οι εγγραφές του ενώνονται με βάση το κοινό admission_id, ώστε κρατήσουμε τις δραστικές ουσίες που συνυπήρξαν στην ίδια νοσηλεία. Με την συνθήκη a1.active_substance < a2.active_substance κρατάμε μόνο τα ζεύγη διαφορετικών δραστικών ουσιών, αποφεύγοντας ταυτόχρονα και εγγραφές τύπου (ασπιρίνη – ασπιρίνη) αλλά και αντεστραμμένα ζεύγη, τύπου (ουσία A – ουσία B) και (ουσία B – ουσία A). Στη συνέχεια ομαδοποιούμε τα δεδομένα με βάση τα ζεύγη δραστικών ουσιών, μετράμε πόσες φορές εμφανίστηκαν, τα ταξινομούμε με βάση συχνότητα εμφάνισης και στο τέλος κρατάμε μόνο τα 3 πιο συχνά ζεύγη.

B11. Βρείτε όλους τους ιατρούς που έχουν εκτελέσει τουλάχιστον 5 λιγότερες επεμβάσεις από τον ιατρό με τις περισσότερες επεμβάσεις στο τρέχον έτος.

```
WITH YearlySurgeries AS (  
    SELECT  
        ma.surgeon_id,  
        COUNT(ma.medical_act_id) AS surgery_count  
    FROM  
        medical_acts ma  
    JOIN  
        admission a ON ma.admission_id = a.admission_id  
    WHERE  
        ma.medical_act_category = 'Χειρουργική'  
        AND EXTRACT(YEAR FROM a.admission_date) = EXTRACT(YEAR FROM CURRENT_DATE)  
    GROUP BY  
        ma.surgeon_id  
)  
MaxSurgeries AS (  
    SELECT MAX(surgery_count) AS max_count  
    FROM YearlySurgeries  
)  
SELECT  
    d.doctor_id AS "ID Ιατρού",  
    p.first_name AS "Όνομα",  
    p.last_name AS "Επώνυμο",  
    ys.surgery_count AS "Αριθμός Χειρουργείων"  
FROM  
    YearlySurgeries ys  
JOIN  
    doctors d ON ys.surgeon_id = d.doctor_id  
JOIN  
    personnel p ON d.doctor_id = p.personnel_id  
CROSS JOIN  
    MaxSurgeries ms  
WHERE  
    ys.surgery_count <= ms.max_count - 5  
ORDER BY  
    "Αριθμός Χειρουργείων" DESC;
```

Αρχικά, φτιάχνουμε το CTE YearlySurgeries στο οποίο κρατάμε και μετράμε μόνο τις χειρουργικές ιατρικές πράξεις του τρέχοντος έτους και ομαδοποιούμε κατά surgeon_id.

Το αποτέλεσμα είναι ένας προσωρινός πίνακας που περιλαμβάνει τους χειρουργούς και τον αριθμό των χειρουργείων που έχουν εκτελέσει. Στη συνέχεια, φτιάχνουμε ένα δεύτερο CTE που ονομάζεται MaxSurgeries το οποίο έχει μοναδική εγγραφή τον χειρουργό με τις περισσότερες επεμβάσεις. Έπειτα, κάνουμε join το πρώτο CTE με τον πίνακα doctors και personnel ώστε να συνδέσουμε το id τους με το ονοματεπώνυμό τους και στη συνέχεια κάνουμε το καρτεσιανό γινόμενο (CROSS JOIN) αυτού του νέου πίνακα με τον MaxSurgeries. Έτσι, δίπλα από την τιμή του επεμβάσεων του κάθε χειρουργού, υπάρχει η τιμή max_count τις οποίες στη συνέχεια συγκρίνουμε και κρατάμε αυτές που ικανοποιούν την ζητούμενη συνθήκη. Τέλος, ταξινομούμε τα αποτελέσματα με φθίνουσα σειρά επεμβάσεων.

B12. Βρείτε τον απαιτούμενο αριθμό προσωπικού ανά τμήμα και ανά βάρδια για συγκεκριμένη εβδομάδα, με ανάλυση ανά υποκλάση προσωπικού (ιατροί ανά ειδικότητα, νοσηλευτές ανά βαθμίδα, διοικητικό προσωπικό ανά ρόλο).

```
SELECT
  s.shift_date AS "Ημερομηνία Βάρδιας",
  s.shift_type AS "Τύπος Βάρδιας",
  p.personnel_type AS "Τύπος Προσωπικού",
  CASE
    WHEN doc.doctor_id IS NOT NULL THEN doc.specialty
    WHEN n.nurse_id IS NOT NULL THEN n.rank
    WHEN ap.admin_id IS NOT NULL THEN ap.duty
    ELSE 'Άγνωστος Ρόλος'
  END AS "Ειδικότητα / Ρόλος",
  COUNT(ps.personnel_id) AS "Αριθμός Προσωπικού"
FROM
  shift s
JOIN
  departments d ON s.department_id = d.department_id
JOIN
  personnel_shifts ps ON s.shift_id = ps.shift_id
JOIN
  personnel p ON ps.personnel_id = p.personnel_id
LEFT JOIN
  doctors doc ON p.personnel_id = doc.doctor_id
LEFT JOIN
  nurses n ON p.personnel_id = n.nurse_id
LEFT JOIN
  administrative_personnel ap ON p.personnel_id = ap.admin_id
WHERE
  s.shift_date BETWEEN '2026-05-01' AND '2026-05-07'
GROUP BY
  d.department_description,
  s.shift_date,
  s.shift_type,
  p.personnel_type,
  "Ειδικότητα / Ρόλος"
ORDER BY
  "Τμήμα",
  "Ημερομηνία Βάρδιας",
  "Τύπος Βάρδιας";
```

Αρχικά, συνδέουμε τους βασικούς πίνακες που θα χρειαστούμε: τις βάρδιες (shift) με τα τμήματα (departments), τις βάρδιες με το προσωπικό που θα τη δουλέψει (personnel_shifts) και στη συνέχεια το προσωπικό με τον πίνακα personnel ώστε να πάρουμε τα υπόλοιπα στοιχεία τους. Μετά, καθώς χρειαζόμαστε διαφορετικές πληροφορίες ανάλογα με τον τύπου προσωπικού, κάνουμε left-join με τους πίνακες doctors, nurses και administrative_personnel. Αυτό μας επιτρέπει να δούμε αν το συγκεκριμένο personnel_id βρίσκεται στον εκάστοτε πίνακα και αν όχι να βάλουμε NULL. Για τον ίδιο λόγο χρησιμοποιούμε το CASE. Ανάλογα με τον τύπο προσωπικού, παίρνουμε και το αντίστοιχο καθήκον/ρόλο. Στη συνέχεια, φιλτράρουμε τις εγγραφές μας ώστε να περιλαμβάνονται σε μία συγκεκριμένη εβδομάδα και μετά ομαδοποιούμε τα αποτελέσματα κατά τμήμα, ημερομηνία, τύπο βάρδιας (πρωί – απόγευμα – βράδυ) και ρόλο. Παράλληλα, μετράμε πόσες τέτοιες ομάδες χρειάζονται και στο τέλος ταξινομούμε κατά τμήμα, μετά κατά ημερομηνία και τέλος κατά τύπο βάρδιας.

B13. Βρείτε για κάθε ιατρό όλη την ιεραρχία εποπτείας του, από τον άμεσο επόπτη έως τον Διευθυντή, με ένδειξη του επιπέδου σε κάθε βαθμίδα.

```
WITH RECURSIVE SupervisionHierarchy AS (
    SELECT
        doctor_id AS original_doctor_id,
        supervisor_id AS current_supervisor_id,
        1 AS hierarchy_level
    FROM
        doctors
    WHERE
        supervisor_id IS NOT NULL

    UNION ALL

    SELECT
        sh.original_doctor_id,
        d.supervisor_id,
        sh.hierarchy_level + 1
    FROM
        SupervisionHierarchy sh
    JOIN
        doctors d ON sh.current_supervisor_id = d.doctor_id
    WHERE
        d.supervisor_id IS NOT NULL
)

SELECT
    sh.original_doctor_id AS "Κωδικός Ιατρού",
    p_doc.last_name AS "Επώνυμο Ιατρού",
    p_doc.first_name AS "Όνομα Ιατρού",
    sh.hierarchy_level AS "Επίπεδο Ιεραρχίας",
    sh.current_supervisor_id AS "Κωδικός Επόπτη",
    p_sup.last_name AS "Επώνυμο Επόπτη",
    p_sup.first_name AS "Όνομα Επόπτη",
    d_sup.rank AS "Βαθμίδα Επόπτη"
FROM
    SupervisionHierarchy sh
JOIN
    personnel p_doc ON sh.original_doctor_id = p_doc.personnel_id
JOIN
    doctors d_sup ON sh.current_supervisor_id = d_sup.doctor_id
JOIN
    personnel p_sup ON sh.current_supervisor_id = p_sup.personnel_id
ORDER BY
    "Κωδικός Ιατρού",
    "Επίπεδο Ιεραρχίας";
```

Σε αυτό το ερώτημα χρησιμοποιήσαμε Recursive CTE, καθώς η ιεραρχία μπορεί να έχει μεταβλητό βάθος και μεγαλύτερο του άμεσου επόπτη. Ο προσωρινός πίνακας SupervisionHierarchy χωρίζεται σε δύο μέρη τα οποία ενώνονται με UNION ALL (λόγω ταχύτητας αντί του UNION): Το πρώτο select εντοπίζει όλους του ιατρούς που έχουν άμεσο επόπτη και ορίζει το επίπεδο ιεραρχίας τους ως 1. Το δεύτερο select εκτελείται επαναλαμβανόμενα και ψάχνει τον νέο επόπτη του προηγούμενου επόπτη. Σε κάθε επανάληψη, το επίπεδο αυξάνεται κατά 1. Η βάση της αναδρομής είναι μόλις το πεδίο του επόπτη να γίνει NULL.

Μετά την ολοκλήρωση της αναδρομής, πραγματοποιούμε τρία διαδοχικά join, ένα με το table personnel για τα στοιχεία του ιατρού, ένα με το table personnel για τα στοιχεία του επόπτη και ένα με το table doctors για την βαθμίδα του ιατρού. Τέλος, ταξινομούμε τα αποτελέσματα πρώτα ανά κωδικό ιατρού και μετά ανά επίπεδο ιεραρχίας, ώστε να μπορούμε να διαβάσουμε όλη την αλυσίδα εποπτείας για κάθε ιατρό ξεχωριστά.

B14. Βρείτε ποιες κατηγορίες ICD-10 διαγνώσεων είχαν τον ίδιο αριθμό εισαγωγών σε δύο συνεχόμενα έτη, με τουλάχιστον 5 περιστατικά ανά έτος.

```
WITH YearlyDiagnosisCounts AS (  
    SELECT  
        a.admission_diagnosis AS icd_id,  
        EXTRACT(YEAR FROM a.admission_date) AS admission_year,  
        COUNT(a.admission_id) AS total_admissions  
    FROM  
        admission a  
    WHERE  
        a.admission_diagnosis IS NOT NULL  
        AND EXTRACT(YEAR FROM a.admission_date) IN (2025, 2026)  
    GROUP BY  
        a.admission_diagnosis,  
        EXTRACT(YEAR FROM a.admission_date)  
    HAVING  
        COUNT(a.admission_id) >= 5  
)  
SELECT  
    y1.icd_id AS "Κωδικός ICD",  
    d.icd_description AS "Περιγραφή Διάγνωσης",  
    y1.admission_year AS "Αρχικό Έτος",  
    y2.admission_year AS "Επόμενο Έτος",  
    y1.total_admissions AS "Αριθμός Νοσηλειών"  
FROM  
    YearlyDiagnosisCounts y1  
JOIN  
    YearlyDiagnosisCounts y2  
    ON y1.icd_id = y2.icd_id  
    AND y1.admission_year = 2025  
    AND y2.admission_year = 2026  
    AND y1.total_admissions = y2.total_admissions  
JOIN  
    diagnosis d ON y1.icd_id = d.icd_id  
ORDER BY  
    "Αριθμός Νοσηλειών" DESC,  
    "Κωδικός ICD";
```

Αρχικά, από τον πίνακα admission κρατάμε τα δεδομένα που μας ενδιαφέρουν στον CTE YearlyDiagnosisCounts: οι διαγνώσεις να είναι από τα έτη 2025 και 2026 και το πλήθος των διαφορετικών περιστατικών να είναι μεγαλύτερο από 5. Στη συνέχεια, κάνουμε αυτόν τον πίνακα self-join και κρατάμε τις εγγραφές οι οποίες έχουν την ίδια διάγνωση, η πρώτη είναι στο 2025 και η δεύτερη στο 2026 και ο αριθμός των περιστατικών να είναι ίδιος. Τέλος, συνδέουμε και τον πίνακα diagnosis ώστε να πάρουμε την περιγραφή της κάθε διάγνωσης, καθώς μέχρι τώρα δουλεύαμε με τα icd_ids τους.

B15. Βρείτε την κατανομή των περιστατικών triage ανά επίπεδο επείγοντος, με μέσο χρόνο αναμονής ανά επίπεδο, ποσοστό περιστατικών που οδήγησαν σε νοσηλεία και κατανομή παραπομπών ανά τμήμα.

```
WITH LevelStats AS (
    SELECT
        emergency_level,
        COUNT(case_id) AS total_cases_in_level,
        AVG(handled_time - arrival_time) AS avg_wait_time,
        ROUND((COUNT(admission_id) * 100.0) / COUNT(case_id), 2) AS admission_percentage
    FROM
        emergency_case
    GROUP BY
        emergency_level
),
DeptDistribution AS (
    SELECT
        ec.emergency_level,
        d.department_description,
        COUNT(ec.case_id) AS cases_referred_to_dept
    FROM
        emergency_case ec
    JOIN
        admission a ON ec.admission_id = a.admission_id
    JOIN
        departments d ON a.department_id = d.department_id
    GROUP BY
        ec.emergency_level,
        d.department_description
)

SELECT
    ls.emergency_level AS "Επίπεδο Επείγοντος",
    ls.total_cases_in_level AS "Συνολικά Περιστατικά",
    ls.avg_wait_time AS "Μέσος Χρόνος Αναμονής",
    ls.admission_percentage AS "Ποσοστό Νοσηλειών (%)",
    dd.department_description AS "Τμήμα Παραπομπής",
    dd.cases_referred_to_dept AS "Περιστατικά ανά Τμήμα"
FROM
    LevelStats ls
LEFT JOIN
    DeptDistribution dd ON ls.emergency_level = dd.emergency_level
ORDER BY
    "Επίπεδο Επείγοντος",
    "Περιστατικά ανά Τμήμα" DESC;
```

Για το τελευταίο ερώτημα, θέλαμε να βγάλουμε μια πλήρη στατιστική εικόνα για το πώς λειτουργεί το τμήμα των Επειγόντων (Triage) στο νοσοκομείο. Επειδή οι υπολογισμοί που ζητήθηκαν είχαν διαφορετικό βαθμό λεπτομέρειας επιλέξαμε να σπάσουμε το query σε δύο CTEs. Στο πρώτο κομμάτι, το LevelStats, δουλέψαμε πάνω στον πίνακα

emergency_case. Ομαδοποιήσαμε τα δεδομένα ανά επίπεδο επείγοντος (1 έως 5) και υπολογίσαμε τα βασικά νούμερα: πόσα περιστατικά είχαμε συνολικά, πόσο χρόνο περίμεναν κατά μέσο όρο οι ασθενείς μέχρι να εξυπηρετηθούν και τι ποσοστό αυτών κατέληξε τελικά σε νοσηλεία. Για το ποσοστό, χρησιμοποιήσαμε τη ROUND πάνω στο αποτέλεσμα της διαίρεσης των εισαγωγών προς τα συνολικά περιστατικά, ώστε να έχουμε μια καθαρή εικόνα με δύο δεκαδικά. Στη συνέχεια, με το DeptDistribution, εστιάσαμε σε όσους ασθενείς παραπέμφθηκαν για νοσηλεία. Εδώ χρειάστηκε να ενώσουμε τα επείγοντα με τις νοσηλείες (admission) και τα τμήματα (departments), ώστε να μετρήσουμε πόσοι ασθενείς από κάθε επίπεδο προτεραιότητας πήγαν σε ποια κλινική. Τέλος, ενώσαμε αυτά τα δύο σύνολα δεδομένων με χρήση του LEFT JOIN. Το κάναμε για να είμαστε σίγουροι ότι αν σε κάποιο επίπεδο επείγοντος έτυχε να μην γίνει καμία νοσηλεία, το επίπεδο αυτό δεν θα εξαφανιζόταν από τα αποτελέσματα, αλλά θα εμφάνιζε κανονικά τα υπόλοιπα στατιστικά του.